VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

WEB APPLICATION DEVELOPMENT
(IT093IU)

FINAL REPORT
INVENTORY MANAGEMENT SYSTEM

GROUP 2 - MEMBER LIST

No.	Full Name	Student ID
1	Lê Thái Minh Tín	ITCSIU24086
2	Huỳnh Ngọc Thủy Tiên	ITITIU24065

Instructor: MSc. Nguyen Trung Nghia

Contents

1 Executive Summary **1**

2 Requirements Analysis **2**
2.1 Problem Statement . . . 2
2.2 User Stories . . . 2
2.3 Functional Requirements . . . 4
2.4 Non-Functional Requirements . . . 4

3 System Architecture **5**
3.1 Architecture Overview . . . 5
3.2 Technology Stack Justification . . . 7
3.3 Architectural & Design Patterns . . . 8
3.4 API Design . . . 8

4 Database Design **10**
4.1 Entity-Relationship Diagram . . . 10
4.2 Table Schemas . . . 11
4.3 Relationships Explanation . . . 13
4.4 Normalization Analysis . . . 14
4.5 Indexing Strategy . . . 14

5 Implementation Details **15**
5.1 Key Algorithms and Business Logic . . . 15
5.2 Security Implementation . . . 17
5.3 Error Handling Strategy . . . 18
5.4 Advanced Features Implementation . . . 19
5.5 AI Assistance Report . . . 20

6 Testing and Quality Assurance **21**
6.1 Testing Strategy . . . 21
6.2 Bug Tracking . . . 22
6.3 Known Limitations . . . 23

7 Deployment **24**
7.1 Deployment Process . . . 24
7.2 CI/CD Pipeline . . . 25
7.3 Monitoring and Maintenance . . . 26

8 User Guide **27**
8.1 Getting Started . . . 27
8.2 Main Feature Walkthroughs . . . 27

9 Team Collaboration **33**
9.1 Team Organization . . . 33
9.2 Individual Contributions . . . 33
9.3 Challenges in Teamwork . . . 34

10 Conclusion and Reflection **34**
10.1 Project Summary . . . 34
10.2 Lessons Learned . . . 34
10.3 Individual Reflections . . . 35

10.4 Future Enhancements 36

Appendices **37**

Appendix A: Meeting Logs **37**

1 Executive Summary

The Inventory Management System is a comprehensive full-stack application developed as the core project for the Web Application Development course. Two team members managed the development process behind this system, the responsibilities of whom are as follows:

- **Le Thai Minh Tin (ITCSIU24086)** served as the Full-stack Developer and Project Manager, responsible for designing and implementing the server infrastructure, database integration, and business logic; as well as deployment, progress tracking and supporting Tien in developing the overall interface of the application.
- **Huynh Ngoc Thuy Tien (ITITIU24065)** served as the Frontend Developer, leading the design and development of the client-facing user interface and overall user experience.

The motivation behind developing this system is grasping how concepts learnt during the course apply to a web application that solves real-world problems and eventually becomes profitable and sustainable. Thus, the team has investigated three existing inventory applications, which are Blue Yonder, OdooWMS and Shopee Management Channel. The results of this investigation revealed that despite the varying complexity of the applications, all of them share the same core features, namely actionable data and a clear management pipeline. This finding has created a baseline for our project to follow within the five-week deadline.

Key Features:

- **Detailed Dashboard Analytics:** A dashboard that provides insights on the current stock ecosystem, profit and capacity of warehouses in the system. This dashboard utilizes bar and line charts to emphasize the actionable insights behind the trends.
- **Durable Data Management:** Full CRUD (Create, Read, Update, Delete) with soft-delete for products, categories, suppliers and warehouses. This includes multipart image upload, SKU tracking generated from the backend and low stock or out of stock indicators generated automatically.
- **Automated Transaction Workflows:** Separate modules to deal with Purchase, Sell and Return processes that automatically update inventory quantities in the whole database immediately after a transaction is done.
- **Proposals & In-App Notifications:** A built-in communication pipeline for staff members to generate purchase requests for depleted inventory that managers can formally approve or reject. This action causes the users involved to receive alerts about new notifications.
- **Role-Based Security and Auditing:** Strict access controls with JWT authentication to distinguish between Administrator, Manager and Staff permissions. A comprehensive administrative audit log is kept to log all user actions to ensure full accountability.

Technology Stack

The application is built on a modern, scalable full-stack architecture utilizing the following technologies:

- **Backend Framework:** Java 21 and Spring Boot 3.3.5
- **Security:** Spring Security with JSON Web Tokens (JWT) for robust authentication
- **Database and Data Access:** MySQL 8+ utilizing raw JDBC repositories (intentionally bypassing JPA to maintain fine-grained SQL control)

- **API Documentation:** RESTful API documented via Springdoc OpenAPI (Swagger)
- **Frontend Architecture:** React 19.2.0 structured as a Single Page Application (SPA)
- **UI and Styling:** Tailwind CSS for responsive design
- **State Management and Networking:** React Query for server state and Axios for data fetching
- **Form Validation:** React Hook Form paired with Zod schemas for strict client-side validation

The system's frontend user interface has been deployed and is actively accessible online.[1] The backend API is also live and operational,[2] with Swagger UI documentation available at a dedicated endpoint.[3]. Initial credentials are injected directly into the system via a database seeding script during the initial setup phase. Once logged in, the Administrator can generate specific credentials for test Managers and Staff members.

2 Requirements Analysis

2.1 Problem Statement

Managing goods constantly face inventory shrinkage, operational bottlenecks, and financial blind spots. This issue is exacerbated across fragmented systems as companies lose track of stock locations, supplier costs and capital tied up in current inventory. Furthermore, without automated alerts, critically low stock levels often go unnoticed until an order cannot be fulfilled, resulting in lost revenue and reputational damage. This system addresses inventory opacity by providing a single source of truth that enforces stock accuracy through logged transactions.

The primary users span three roles. First, the **staff** need fast, reliable interfaces to log stock movements in real-time, quickly locate product details and alert management about restocking needs. Second, **managers** require real-time data visibility when reviewing inventory values or transaction histories and a digital workflow to replace paper-based purchase approvals. Finally, **administrators** need role-based access control to prevent unauthorized access to sensitive data and a clear audit trail for accountability for all users in the system.

2.2 User Stories

Must Have

1. As an **Admin**, I want to create, view, and manage user accounts, so that only authorised company personnel can access the application.
2. As a **User**, I want to log in securely using a username and password (JWT), so that my data and session are protected from unauthorised access.
3. As a **Manager**, I want to process Return-to-Supplier transactions to send defective or excess inventory back to vendors, so that stock levels are accurately decremented and the supplier relationship is recorded in the transaction history.
4. As a **Manager**, I want to create and manage warehouse locations, so that the system accurately reflects where physical inventory is stored.

[1] https://inventory-management-sigma-puce.vercel.app
[2] https://inventory-management-production-6424.up.railway.app/api
[3] https://inventory-management-production-6424.up.railway.app/swagger-ui/index.html

5. As a **Staff member**, I want to add new products to the catalogue, so that incoming new inventory can be tracked by the system.
6. As a **Staff member**, I want the system to automatically generate a unique product code (e.g. `SKU000001`) for new items, so that manual data-entry errors are eliminated.
7. As a **Staff member**, I want to record "Purchase" transactions, so that stock quantities are automatically incremented when shipments arrive.
8. As a **Staff member**, I want to record "Sell" transactions, so that the database automatically decrements current stock levels appropriately when goods leave the warehouse.
9. As a **Staff member**, I want to see clear low-stock and out-of-stock indicators on the product list, so that I immediately know which items need attention.
10. As a **Manager**, I want to receive purchase proposals submitted by staff, so that I can formally approve or reject expenditures before items are ordered.
11. As a **User**, I want to search, sort, and filter the products and transactions lists, so that I can quickly find specific records in a large database.
12. As a **User**, I want to soft-delete (toggle active/inactive) categories and suppliers rather than permanently deleting them, so that historical transaction data remains intact and accurate.
13. As a **User**, I want to navigate through lists using pagination, so that the application loads quickly even when dealing with thousands of product records.

Should Have

1. As a **Staff member**, I want to upload multipart images for products, so that warehouse workers can visually identify items on the floor to avoid mix-ups.
2. As a **Manager**, I want to view a centralised dashboard with summary metrics and transaction charts, so that I can instantly evaluate the financial health of the inventory.
3. As a **User**, I want to receive in-app notifications with unread counts, so that I am instantly alerted when my proposal is approved or when a transaction requires attention.
4. As a **Staff member**, I want to link products to specific preferred suppliers along with their unit costs, so that future purchasing decisions are financially optimised.
5. As an **Admin**, I want to view an uneditable audit log of system actions, so that I can trace any data anomalies back to a specific user and timestamp.
6. As a **Manager**, I want to export product lists, dashboard stats, and transaction histories to CSV files, so that I can use the data in external spreadsheet software for corporate reporting.
7. As a **User**, I want to manage my personal profile and update my password, so that I can take personal responsibility for my account security.

2.3 Functional Requirements

Authentication and User Management

The system must provide a secure authentication gateway that validates user credentials against the database and issues a secure JWT for session management. The backend API endpoints and frontend routes must be strictly based on the user's assigned role (ADMIN, MANAGER, STAFF). Moreover, an administrative interface must be created for creation and management of these user accounts.

Product and Master Data Control

The system must support complex product data entry, automatically generating fixed SKU identifiers upon creation. The products must be linkable to actively managed Categories, Suppliers, and Warehouses. Besides, the system must accept multipart form data for product image uploads. Furthermore, all deletions must be implemented as soft-deletes using a status flag, preserving historical data integrity.

Transaction Processing and Workflow Automation

The system must provide a transaction engine that can handle Purchase, Sell and Return transactions. Upon successful logging of any transaction, the stock quantity of the product in question must be automatically recalculated. To support purchasing, Staff must be able to create a proposal which is sent to Managers to approve or reject. Any change to a proposal's status should automatically send a push notification to the corresponding user.

2.4 Non-Functional Requirements

Performance

The application must be highly responsive to ensure warehouse operations are not delayed. Standard API queries, such as fetching the product list or searching for a specific SKU, should execute and render on the client side within 500 milliseconds under normal load. Heavier operations, such as generating the multi-metric dashboard charts or processing large CSV file exports, must complete within 2 to 3 seconds.

Security

All user passwords must be securely hashed and salted in the MySQL database (using BCrypt). All API requests must require a valid JWT passed in the Authorization header. Uploaded files must be strictly validated to ensure only image formats are accepted, with a hard file size limit (5 MB enforced at the application layer) to prevent malicious payloads or server storage exhaustion. Furthermore, Cross-Origin Resource Sharing (CORS) must be strictly configured to only allow requests from the official frontend domain.

Usability

The frontend interface must be intuitive and require minimal training for warehouse staff. It must be built with responsive design principles so that it functions seamlessly on standard desktop monitors in the manager's office as well as on tablets utilized by staff on the warehouse floor. The UI must provide immediate, clear feedback through smooth loading states and human-readable form validation errors.

Scalability

While initially deployed for a single facility, the system architecture must be designed to scale. The backend, utilizing Spring Boot and raw JDBC for highly optimized queries, should easily support up to 100-200 concurrent internal users without performance degradation. The database schema must be properly normalized and indexed (especially on foreign keys and search columns like product names and SKUs) to ensure that query performance remains acceptable as historical transaction volume grows over time.

Reliability

The system must guarantee high data integrity and maximum uptime during operational warehouse hours. Implementing soft-deletion across all major entities ensures that accidental

clicks do not result in catastrophic data loss or broken relational links in historical transaction logs. Furthermore, the backend must feature global exception handling to gracefully catch database timeouts or bad requests, returning standard error envelopes rather than crashing the server application.

3 System Architecture

3.1 Architecture Overview

The Inventory Management System is built on a REST API backend with a layered architecture on the server side. This system adopts MVC principles where the React frontend acts as the View, the Spring Boot Controller layer maps to the Controller, and the Service and Repository layers collectively form the Model.

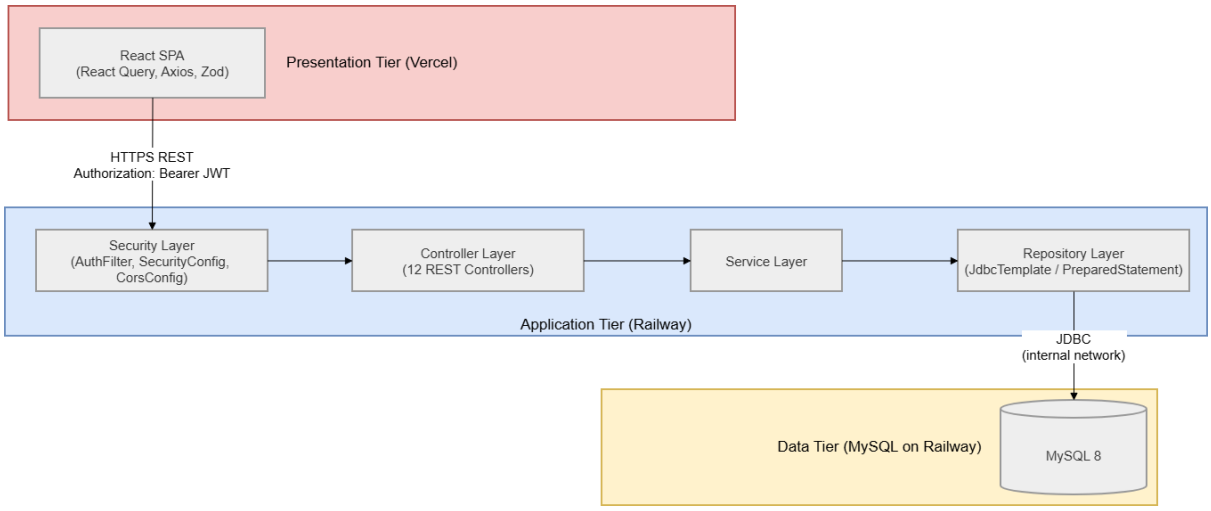


Figure 1: Architecture Diagram of the system

High-Level Components

- **Presentation Tier:** A Single-Page Application running in the user's browser that communicates exclusively with the backend through RESTful HTTP calls.
- **Application Tier:** A stateless REST API that enforces all business rules, validates requests, manages authentication and authorization, and orchestrates database operations.
- **Data Tier:** A relational database storing all persistent application data across 12 normalized tables and is accessed exclusively by the backend.

Backend Layered Architecture

The Spring Boot backend is organized into four distinct layers, each with a clear responsibility:

1. **Controller Layer** (`controller/`): A system of twelve controllers receives HTTP requests, performs input validation using Jakarta Bean Validation (`@Valid`) and delegates to the service layer. All responses are wrapped in the unified `Response` envelope class and returned as JSON.
2. **Service Layer** (`service/` and `service/Impl/`): Services are defined as interfaces with a single `Impl` implementation class. Key services are for transactions (stock mutation), proposals (approval workflow), products and notifications.

3. **Repository Layer** (`repository/` and `repository/Impl/`): Abstracts all database access behind Java interfaces, using raw JDBC with either Spring's `JdbcTemplate` or raw `Connection`/`PreparedStatement`.

4. **Security Layer** (`security/`): Spring Security validates every incoming request. Then, `AuthFilter` extracts and verifies the JWT from the `Authorization` header. Next, `SecurityConfig` maps URL patterns to required role and `CorsConfig` restricts allowed origins to the configured frontend domain.

Data Flow

The sequence of a typical authenticated request (e.g., a Manager creating a purchase transaction) is described in the diagram below:

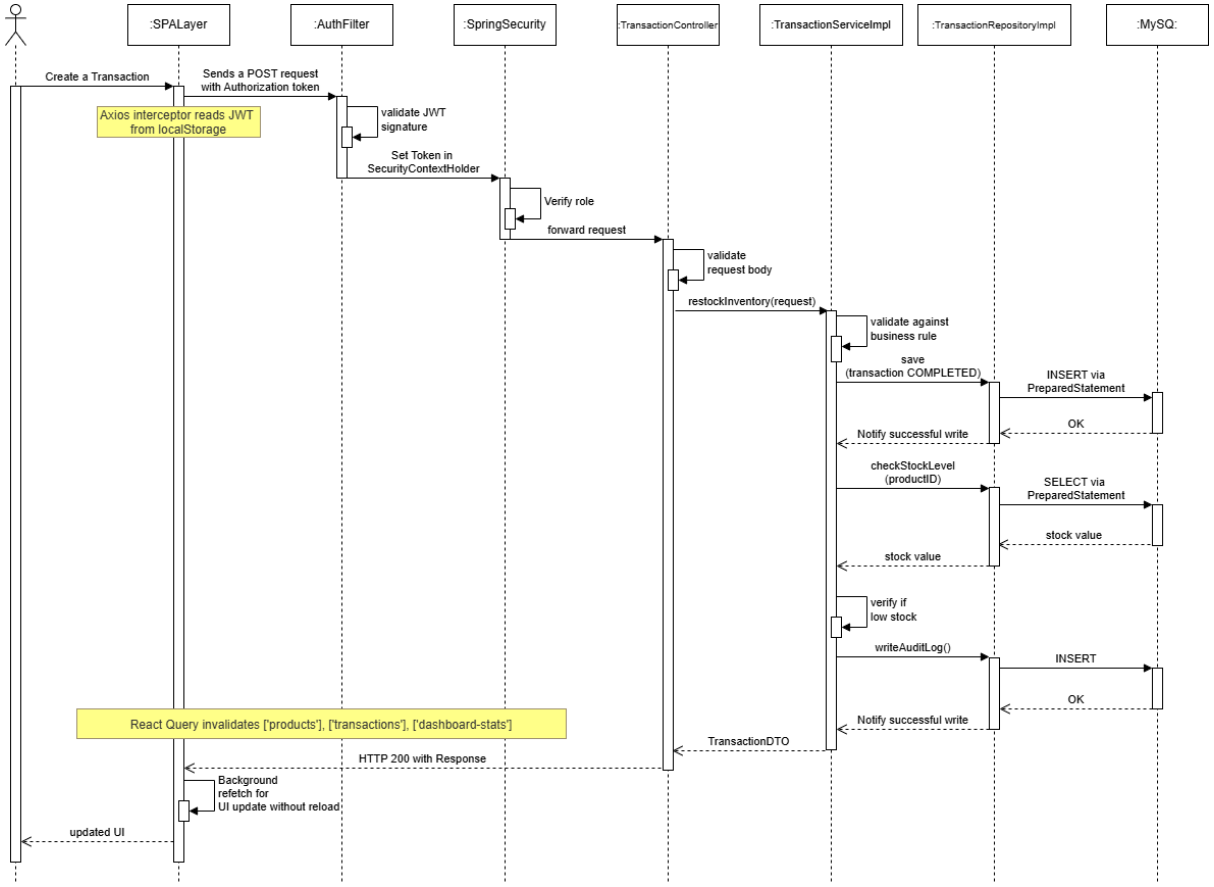


Figure 2: Sequence Diagram of the data flow

The key points of this diagram are:

1. **JWT Attachment in Frontend:** The React frontend reads the JWT from `localStorage` and attaches it as an `Authorization: Bearer` header via an Axios request interceptor, before the request leaves the browser.

2. **JWT Validation:** The request arrives at Spring Boot and passes through `AuthFilter`, which validates the token's signature and expiry using `JwtUtils`. On success, a legitimate token is placed in the `SecurityContextHolder`.

3. **Role Authorization:** Spring Security's `authorizeHttpRequests` rules verify that the authenticated user holds the `MANAGER` role required for the requested endpoint. Thus,

requests failing either JWT validation or role checks are rejected here and never reach business logic.

4. **Input Validation (Controller Layer):** `TransactionController` receives the forwarded request and applies Jakarta Bean Validation (`@Valid`) to the request body. The controller delegates immediately to the service layer, holding no business logic of its own.

5. **Business Logic Orchestration (Service Layer):** The method `restockInventory()` performs all domain operations in sequence: validating business rules, incrementing the product stock, persisting a `COMPLETED` transaction record, evaluating stock levels to fire low-stock notifications where applicable, and writing an audit log entry.

6. **Persistence (Repository Layer):** Each database operation is executed via JDBC, preventing SQL injection at the data access boundary. The repository returns plain data objects so no SQL leaks into the service layer.

7. **Response and Cache Invalidation:** The controller wraps the result in the unified `Response` envelope and returns HTTP 200. On receipt, React Query on the frontend invalidates the `['products']`, `['transactions']`, and `['dashboard-stats']` cache keys, triggering automatic background refetches that update the UI without a full page reload.

3.2 Technology Stack Justification

Frontend: React 19 over Vue 3 and Angular 17

React was chosen for its large ecosystem and the flexibility of its component model. Compared to Angular, which imposes a rigid architecture, React allowed the team to iterate rapidly on UI features. Meanwhile, React's libraries are more suited for data-heavy CRUD dashboards than Vue 3. On the other hand, React Query was a decisive factor with its server-state caching, `invalidateQueries` API, and stale-while-revalidate behavior that perfectly match the requirement of keeping live inventory data fresh after every mutation without manual state management.

Backend: Spring Boot 3.3.5 over Node.js and Django

Spring Boot was selected for its strong type system, which prevents entire categories of runtime bugs, and for Spring Security, which is one of the most reliable frameworks for authorization and authentication. Compared to Django REST Framework, Spring Boot's explicit constructor-based dependency injection produces more testable code. As for NestJS, Spring Boot's JDBC support gives finer control over database operations, which was a deliberate decision.

Database: MySQL 8 over PostgreSQL and MongoDB

MySQL was chosen over PostgreSQL because both team members had prior experience with it, and the system's data model is fundamentally relational. MongoDB was ruled out because a document store would have made multi-table joins and referential integrity unnecessarily complex.

Raw JDBC over Spring Data JPA

JPA's automatic schema generation and lazy-loading behavior were intentionally bypassed, because raw JDBC gives complete control over every SQL statement, prevents the N+1 query problem common in JPA-based systems and makes query performance transparent. Furthermore, dynamic multi-filter queries (e.g., products filtered simultaneously by category, warehouse, price range, stock status, and a search term) are far easier to construct correctly using a `StringBuilder` SQL builder than with JPA's `Specification` API.

Hosting: Vercel (Frontend) and Railway (Backend and Database)

The frontend is deployed on Vercel, whose GitHub integration redeploys the SPA automatically on every push to the `master` branch with zero configuration. Additionally, the backend and database are hosted on Railway, which provisions the JVM service and MySQL within

the same private network, reducing latency and avoiding unnecessary external exposure. All sensitive configuration (database credentials, JWT secret, CORS allowed origins) is injected as Railway environment variables so no secrets are hard-coded.
Besides, the backend was initially self-hosted on a RackNerd VPS running Ubuntu, with the Spring Boot process managed by `systemd` and reverse-proxied through Nginx. While this approach provided full control over the server environment, it introduced connection instability concerns due to the VPS being in the US. The stack was migrated to Railway mid-project, which eliminated this overhead while preserving the same environment variable injection pattern for secrets.

3.3 Architectural & Design Patterns

Model-View-Controller (MVC): The backend follows a strict layered MVC structure where controllers parse requests and delegate immediately to services, which then handle all business logic. Finally, all SQL lives in the repository layer, so no logic crosses layer boundaries.

Repository Pattern: Every entity has a Java interface declaring its data-access operations with a concrete implementation. Consequently, services depend only on the interface, so the underlying access strategy can change without touching callers, and repositories can be mocked in unit tests.

Singleton Pattern: Spring's IoC container manages every `@Service`, `@Repository`, and `@Component` as a singleton. For example, `NotificationStreamServiceImpl` relies on this explicitly as its `ConcurrentHashMap` of active SSE emitters is an instance field that is only correct when shared as a single instance across all threads.

Template Method Pattern: `TransactionServiceImpl` exposes three public methods (`restockInventory`, `sell`, `returnToSupplier`) that share an identical skeleton: validate inputs, check stock, redirect `STAFF` to the proposal workflow, update stock, record the transaction and write an audit log. Invariant steps are factored into private helpers shared across all three methods.

Strategy Pattern: `ProposalServiceImpl.executeProposal()` dispatches to a different stock-update path based on the `TransactionType` enum (`PURCHASE`, `SALE`, `RETURN_TO_SUPPLIER`), selecting the correct algorithm at runtime without an inheritance hierarchy.

3.4 API Design

RESTful Principles

The API follows REST conventions: resources are nouns in the URL path, HTTP verbs encode the operation, and the server is stateless (every request carries its JWT). All resource URLs are prefixed with `/api/` and use lowercase, hyphen-separated names.

URL Structure

Pattern	Example	Purpose
`GET /api/{res}/all`	`/api/products/all`	Paginated list
`GET /api/{res}/{id}`	`/api/products/5`	Single resource
`POST /api/{res}/add`	`/api/categories/add`	Create
`PUT /api/{res}/update/{id}`	`/api/suppliers/update/3`	Full update
`DELETE /api/{res}/delete/{id}`	`/api/users/delete/7`	Soft delete
`PUT /api/{res}/toggle-status/{id}`	`/api/categories/toggle-status/2`	Activate/deactivate
`GET /api/{res}/export`	`/api/transactions/export`	CSV download

The current URL structure omits an explicit version prefix (e.g., `/api/v1/`). Since the frontend and backend are developed and deployed together as a single system without external API consumers, versioning was not a priority for this iteration. In a production system with

third-party clients, a version prefix would be introduced to allow breaking changes without disrupting existing integrations.

Unified Response Envelope

Every endpoint (except `GET /api/users/current`) returns the same `Response` class serialized to JSON. The `@JsonInclude(NON_NULL)` annotation omits all unpopulated fields, keeping payloads lean. Example paginated product list response:

```
{
  "status": 200,
  "message": "success",
  "products": [{ "id": 1, "name": "Widget A", "sku": "SKU000001", ... }],
  "totalPages": 5,
  "totalElements": 47,
  "timestamp": "2025-05-31T10:22:00"
}
```

HTTP Status Code Strategy

- **200 OK** for successful operation. Business-level failures surface in `message` with a non-200 `status` field.
- **400 Bad Request** for validation failure or business constraint violation (e.g., insufficient stock, duplicate email).
- **401 Unauthorized** for invalid, expired, or missing JWT (`CustomAuthenticationEntryPoint`).
- **403 Forbidden** for valid JWT but insufficient role (`CustomAccessDenialHandler`).
- **404 Not Found** for (`NotFoundException`).
- **429 Too Many Requests** for in-memory rate limiter triggered on `POST /api/auth/login` (max 10 attempts per 60 seconds per IP).
- **500 Internal Server Error** for unhandled exception caught by `GlobalExceptionHandler`.

4 Database Design

4.1 Entity-Relationship Diagram

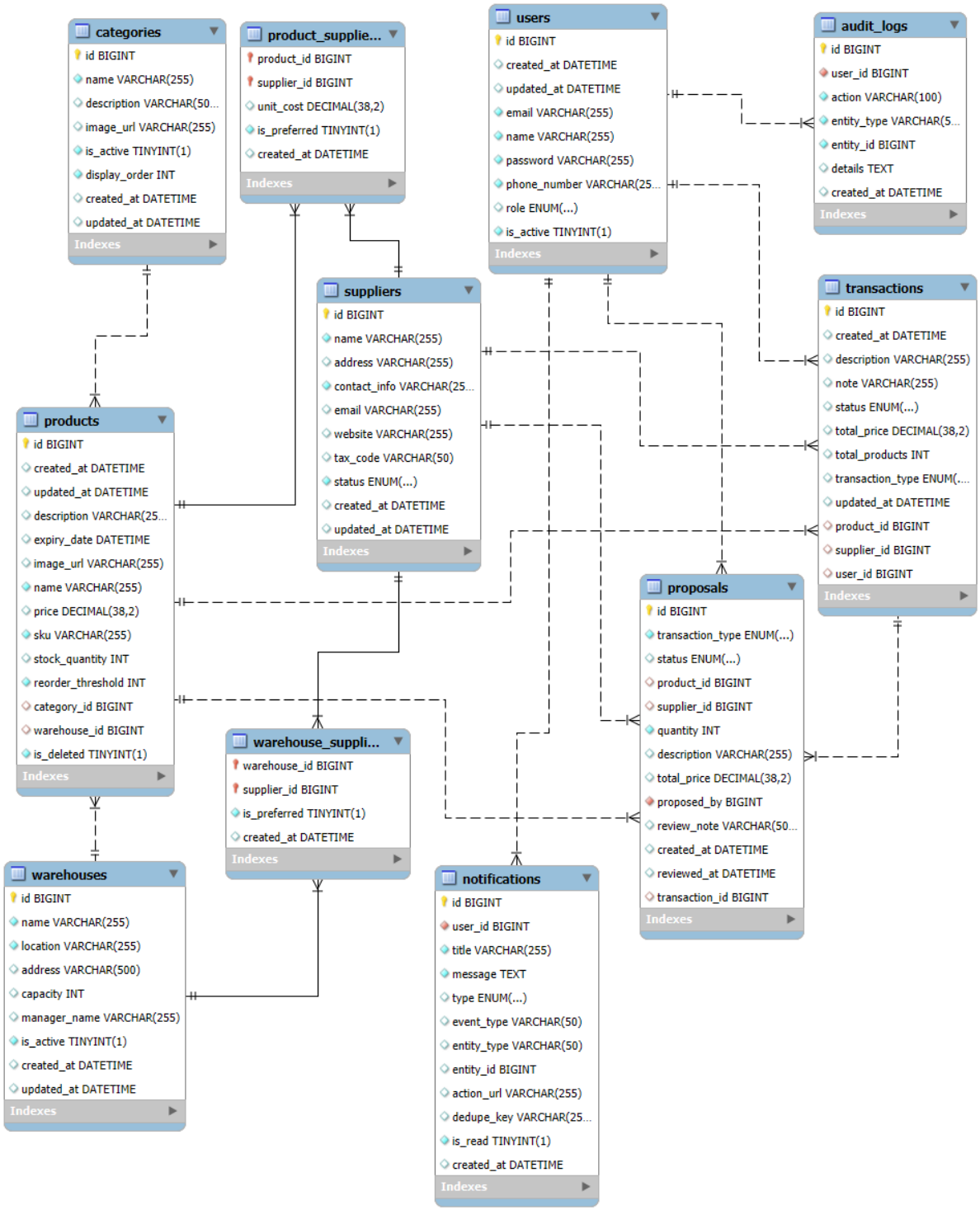


Figure 3: Simplified ERD with audit columns and their respective foreign relationships omitted for readability.

4.2 Table Schemas

```
Table: users
- id           BIGINT          PK AUTO_INCREMENT
- name         VARCHAR(255)    NOT NULL
- email        VARCHAR(255)    UNIQUE NOT NULL
- password     VARCHAR(255)    NOT NULL  (BCrypt hash)
- phone_number VARCHAR(255)    NOT NULL
- role         ENUM(ADMIN,MANAGER,STAFF)
- is_active    BOOLEAN         NOT NULL DEFAULT TRUE
- created_at   DATETIME
- updated_at   DATETIME
- created_by   BIGINT          FK -> users(id) ON DELETE SET NULL
- updated_by   BIGINT          FK -> users(id) ON DELETE SET NULL

Table: categories
- id            BIGINT         PK AUTO_INCREMENT
- name          VARCHAR(255)   NOT NULL
- description   VARCHAR(500)
- image_url     VARCHAR(255)
- is_active     BOOLEAN        NOT NULL DEFAULT TRUE
- display_order INT            NOT NULL DEFAULT 0
- created_at    DATETIME       DEFAULT CURRENT_TIMESTAMP
- updated_at    DATETIME       DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
- created_by    BIGINT         FK -> users(id) ON DELETE SET NULL
- updated_by    BIGINT         FK -> users(id) ON DELETE SET NULL

Table: warehouses
- id           BIGINT          PK AUTO_INCREMENT
- name         VARCHAR(255)    NOT NULL
- location     VARCHAR(255)    NOT NULL
- address      VARCHAR(500)
- capacity     INT
- manager_name VARCHAR(255)
- is_active    BOOLEAN         NOT NULL DEFAULT TRUE
- created_at   DATETIME        DEFAULT CURRENT_TIMESTAMP
- updated_at   DATETIME        DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
- created_by   BIGINT          FK -> users(id) ON DELETE SET NULL
- updated_by   BIGINT          FK -> users(id) ON DELETE SET NULL

Table: suppliers
- id           BIGINT          PK AUTO_INCREMENT
- name         VARCHAR(255)    NOT NULL
- address      VARCHAR(255)
- contact_info VARCHAR(255)    NOT NULL
- email        VARCHAR(255)
- website      VARCHAR(255)
- tax_code     VARCHAR(50)
- status       ENUM(ACTIVE,INACTIVE) NOT NULL DEFAULT 'ACTIVE'
- created_at   DATETIME        DEFAULT CURRENT_TIMESTAMP
- updated_at   DATETIME        DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
- created_by   BIGINT          FK -> users(id) ON DELETE SET NULL
```

```
- updated_by   BIGINT          FK -> users(id) ON DELETE SET NULL

Table: products
- id                BIGINT          PK AUTO_INCREMENT
- name              VARCHAR(255)    NOT NULL
- sku               VARCHAR(255)    UNIQUE NOT NULL
- price             DECIMAL(38,2)
- stock_quantity    INT             CHECK (stock_quantity >= 0)
- reorder_threshold INT             NOT NULL DEFAULT 10
- category_id       BIGINT          FK -> categories(id) ON DELETE CASCADE
- warehouse_id      BIGINT          FK -> warehouses(id) ON DELETE SET NULL
- description       VARCHAR(255)
- expiry_date       DATETIME
- image_url         VARCHAR(255)
- is_deleted        BOOLEAN         NOT NULL DEFAULT FALSE
- created_at        DATETIME
- updated_at        DATETIME
- created_by        BIGINT          FK -> users(id) ON DELETE SET NULL
- updated_by        BIGINT          FK -> users(id) ON DELETE SET NULL

Table: product_suppliers  (junction table)
- product_id   BIGINT          PK, FK -> products(id) ON DELETE CASCADE
- supplier_id  BIGINT          PK, FK -> suppliers(id) ON DELETE CASCADE
- unit_cost    DECIMAL(38,2)
- is_preferred BOOLEAN         NOT NULL DEFAULT FALSE
- created_at   DATETIME        DEFAULT CURRENT_TIMESTAMP

Table: transactions
- id               BIGINT          PK AUTO_INCREMENT
- transaction_type ENUM(PURCHASE,SALE,RETURN_TO_SUPPLIER)
- status           ENUM(PENDING,PROCESSING,COMPLETED,CANCELLED)
- total_products   INT
- total_price      DECIMAL(38,2)
- description      VARCHAR(255)
- note             VARCHAR(255)
- product_id       BIGINT          FK -> products(id) ON DELETE SET NULL
- supplier_id      BIGINT          FK -> suppliers(id) ON DELETE SET NULL
- user_id          BIGINT          FK -> users(id) ON DELETE SET NULL
- created_at       DATETIME
- updated_at       DATETIME
- created_by       BIGINT          FK -> users(id) ON DELETE SET NULL
- updated_by       BIGINT          FK -> users(id) ON DELETE SET NULL

Table: proposals
- id               BIGINT          PK AUTO_INCREMENT
- transaction_type ENUM(PURCHASE,SALE,RETURN_TO_SUPPLIER) NOT NULL
- status           ENUM(PENDING,APPROVED,REJECTED) DEFAULT 'PENDING'
- product_id       BIGINT          FK -> products(id) ON DELETE SET NULL
- supplier_id      BIGINT          FK -> suppliers(id) ON DELETE SET NULL
- quantity         INT             NOT NULL
- description      VARCHAR(255)
```

```
- total_price      DECIMAL(38,2)
- proposed_by      BIGINT          NOT NULL, FK -> users(id) ON DELETE RESTRICT
- reviewed_by      BIGINT          FK -> users(id) ON DELETE SET NULL
- review_note      VARCHAR(500)
- created_at       DATETIME        DEFAULT CURRENT_TIMESTAMP
- reviewed_at      DATETIME
- transaction_id   BIGINT          FK -> transactions(id) ON DELETE SET NULL

Table: notifications
- id          BIGINT          PK AUTO_INCREMENT
- user_id     BIGINT          NOT NULL, FK -> users(id) ON DELETE CASCADE
- title       VARCHAR(255)    NOT NULL
- message     TEXT            NOT NULL
- type        ENUM(INFO,WARNING,ERROR,SUCCESS) DEFAULT 'INFO'
- event_type  VARCHAR(50)
- entity_type VARCHAR(50)
- entity_id   BIGINT
- action_url  VARCHAR(255)
- dedupe_key  VARCHAR(255)
- is_read     BOOLEAN         NOT NULL DEFAULT FALSE
- created_at  DATETIME        DEFAULT CURRENT_TIMESTAMP

Table: warehouse_suppliers  (junction table)
- warehouse_id BIGINT         PK, FK -> warehouses(id) ON DELETE CASCADE
- supplier_id  BIGINT         PK, FK -> suppliers(id)  ON DELETE CASCADE
- is_preferred BOOLEAN        NOT NULL DEFAULT FALSE
- created_at   DATETIME       DEFAULT CURRENT_TIMESTAMP
```

4.3 Relationships Explanation

One-to-Many Relationships

- **categories → products (1:N):** Each product belongs to exactly one category. `ON DELETE CASCADE` was chosen because a product is not meaningful without a category; removing a category should remove all its orphaned products.
- **warehouses → products (1:N):** A product is optionally stored in one warehouse (`ON DELETE SET NULL`), preserving product records if a warehouse is removed.
- **users → transactions (1:N):** Each transaction is associated with one user (the operator who created it). `ON DELETE SET NULL` retains the transaction history even if the user account is deactivated.
- **products → transactions (1:N):** Each transaction references one product. `ON DELETE SET NULL` preserves historical transaction records if a product is soft-deleted and then permanently removed.
- **users → proposals (1:N, proposed_by):** Each proposal is submitted by one staff member. `ON DELETE RESTRICT` prevents a user from being deleted while they have outstanding proposals, protecting the approval workflow.
- **users → notifications (1:N):** Notifications are delivered to one user. `ON DELETE CASCADE` removes a user's notifications when their account is deleted.

- **users → audit_logs (1:N):** Each audit log entry references one user. `ON DELETE RESTRICT` ensures audit records are never silently orphaned.

Many-to-Many Relationships

- **product_suppliers:** A product can be supplied by many suppliers, and a supplier can supply many products. The junction table physically stores only the per-relationship `unit_cost` and `is_preferred` flag. Running totals for purchased and returned quantities are derived at query time via a `LEFT JOIN` subquery over the `transactions` table, rather than being stored as separate columns.

- **warehouse_suppliers:** A warehouse can source from many suppliers and a supplier can serve many warehouses. Stores the `is_preferred` flag per relationship.

Self-Referential Relationships

The `users` table references itself via `created_by` and `updated_by` foreign keys, recording which administrator created or last modified each user account. Both use `ON DELETE SET NULL` to prevent cascading deletions when an admin account is removed.

4.4 Normalization Analysis

First Normal Form (1NF): All tables have a single-valued primary key, all attributes are atomic, and there are no repeating groups. The `product_suppliers` table uses a composite PK (`product_id, supplier_id`) rather than storing a comma-separated list of supplier IDs on the `products` table.

Second Normal Form (2NF): In the junction tables, every non-key attribute depends on the full composite primary key. For example, in `product_suppliers`, `unit_cost` and `is_preferred` depend on the (`product_id`, `supplier_id`) pair, not on either key column alone.

Third Normal Form (3NF): No non-key attribute is transitively dependent on the primary key via another non-key attribute. For example, a product's category name is not stored in `products`; only the `category_id` foreign key is stored, and the name is fetched from the `categories` table on demand. Similarly, supplier contact details are not duplicated into `transactions`; only `supplier_id` is stored.

The schema satisfies Third Normal Form (3NF) throughout. Nonetheless, two minor denormalization decisions were made consciously:

- `proposals.total_price` is computed at proposal creation time (quantity × product price) and stored rather than always recalculated. This snapshot preserves the agreed price even if the product price changes later.

- `product_suppliers` total quantities (`total_purchased`, `total_returned`) are not stored as physical columns. Instead, they are derived at query time via a `LEFT JOIN` subquery that aggregates `COMPLETED` transactions for each (`product_id`, `supplier_id`) pair. This keeps the schema simpler by avoiding update maintenance on every transaction, at the cost of a computed aggregation on each lookup. This trade-off is acceptable given the low frequency of product-supplier detail queries.

4.5 Indexing Strategy

The following indexes were defined in `inventory_db.sql` to support the system's most frequent query patterns:

Table	Indexed Column(s)	Justification
products	is_deleted	Every list query filters on soft-delete flag
	name, sku, description	Composite cover for LIKE search on all three fields
transactions	status	Filter by PENDING / COMPLETED / CANCELLED
	transaction_type	Filter by PURCHASE / SALE / RETURN
	created_at	Default sort column for transaction list
	product_id	FK join when loading transaction product details
	supplier_id	FK join performance
	user_id	FK join performance
suppliers	status	Filter supplier list by ACTIVE / INACTIVE
product_suppliers	supplier_id	FK join on junction table
categories	display_order	Default sort for category list
notifications	user_id, is_read	Covers the unread-count query
	user_id, dedupe_key, is_read, created_at	Prevents duplicate alert dispatch
	user_id, type, created_at	Paginated list with type filter
proposals	status	Pending proposals count badge
	proposed_by	Proposals filtered by user
audit_logs	user_id	Audit log filtered by actor
	entity_type, entity_id	Audit log filtered by affected entity
	created_at	Default sort for audit log list

All foreign key columns that were not already covered by a primary key or unique constraint were given explicit indexes. MySQL does not automatically create indexes on the non-leading side of foreign keys, so without these indexes, every FK join (e.g., `transactions JOIN products ON product_id`) would require a full table scan of the transactions table.

5 Implementation Details

5.1 Key Algorithms and Business Logic

Transaction Stock Engine

The core business requirement is that stock quantities are atomically updated when a transaction is recorded. The `TransactionServiceImpl` implements a role-aware dispatch to enforce the approval hierarchy without exposing separate endpoints when a `STAFF` user initiates a purchase or sale, the request is silently redirected to the proposal workflow rather than executing immediately. This ensures that no stock mutation can occur without Manager authorisation, regardless of how the request is constructed.

```
User user = userService.getCurrentLoggedInUser();
```

```
// STAFF cannot execute purchases directly - redirect to proposals
if (user.getRole() == UserRole.STAFF) {
    transactionRequest.setTransactionType(TransactionType.PURCHASE);
    return proposalService.createProposal(transactionRequest);
}
// MANAGER/ADMIN: increment stock and record COMPLETED transaction
product.setStockQuantity(currentStock + quantity);
productRepository.update(product);
transaction.setStatus(TransactionStatus.COMPLETED);
transactionRepository.save(transaction);
```

Listing 1: Role-aware dispatch in restockInventory()

For sale and return transactions, a pre-condition check via `assertSufficientStock()` prevents stock from dropping below zero at the application layer. Then, the `CHECK (stock_quantity >= 0)` constraint in MySQL ensures stock integrity.

Server-Sent Events (SSE) Notification Streaming

Rather than requiring clients to poll for new notifications on a fixed interval which wastes bandwidth and introduces latency, the backend pushes events to connected clients using the HTML5 Server-Sent Events protocol. SSE was chosen over WebSockets because notification delivery is strictly unidirectional (server-to-client); the added complexity of a full-duplex WebSocket connection is unnecessary for this use case and less compatible with standard HTTP/2 infrastructure.

`NotificationStreamServiceImpl` maintains a `ConcurrentHashMap` that connects user IDs to their active browser connections. These thread-safe collections are necessary because multiple concurrent HTTP requests, each opening an SSE connection, may read and write the emitter list simultaneously. Thus, a scheduled heartbeat runs every 20 seconds to detect and remove dead connections; the decision behind which is made to fall within common browser and proxy timeout thresholds before an idle connection is silently dropped.

```
private void publish(Long userId, String eventName, Object payload) {
    List<SseEmitter> emitters = emittersByUserId.get(userId);
    for (SseEmitter emitter : emitters) {
        synchronized (emitter) {
            emitter.send(SseEmitter.event().name(eventName).data(payload));
        }
    }
}
@Scheduled(fixedDelay = 20_000)
public void sendHeartbeat() {
    emittersByUserId.forEach((userId, emitters) -> {
        for (SseEmitter e : emitters)
            e.send(SseEmitter.event().comment("keep-alive"));
    });
}
```

Listing 2: Publishing a notification event to all browser connections

The React frontend subscribes to the `GET /api/notifications/stream` endpoint via a Notification Stream hook using the browser's native `EventSource` API. On receiving a `notification.created` event, React Query's `invalidateQueries(['notifications'])` is triggered, causing the bell icon unread count to update in real time. This approach avoids maintaining a separate client-side notification state that could diverge from the server.

Proposal Approval Workflow

The proposal system enforces the requirement that Staff-initiated stock mutations are reviewed before affecting inventory. A linked transaction record is created immediately at `PENDING` status alongside the proposal so that Staff can track their own requests within the unified transaction list without requiring a separate view.

When a Staff user submits a purchase or sale request, `ProposalServiceImpl.createProposal()` performs three atomic operations:

1. Creates a `proposals` record with `status = PENDING`.
2. Creates a linked `transactions` record with `status = PENDING`, giving the Staff member immediate visibility of their request within the transaction list.
3. Sends an in-app notification to every user with the `MANAGER` role, ensuring timely review.

When a Manager approves, `executeProposal()` updates the product stock, marks the linked transaction as `COMPLETED`, and sends a `SUCCESS` notification to the original proposer. When rejected, the linked transaction is marked `CANCELLED` and an `INFO` notification is sent, maintaining a complete audit trail of both outcomes.

5.2 Security Implementation

Password Hashing

All passwords are hashed with BCrypt via Spring Security's `BCryptPasswordEncoder` that makes brute-force and rainbow-table attacks computationally infeasible. Passwords are never stored or logged in plaintext.

SQL Injection Prevention

All database queries use either positional `PreparedStatement` parameters or Spring `JdbcTemplate`'s parameterised query methods, ensuring user-supplied input is never concatenated directly into SQL. Dynamically constructed `ORDER BY` clauses additionally use a server-side column whitelist, preventing a malicious `sortBy` parameter from injecting arbitrary SQL fragments.

JWT Authentication

Tokens are signed with HMAC-SHA256 using a 256-bit secret key loaded from an environment variable at startup, expire after six months and contain only the user's email. This extended expiry is an acknowledged trade-off: without a token blacklist or refresh token infrastructure, short-lived tokens would require frequent re-authentication, although sessions are expected to be long-running. On every request, `AuthFilter` extracts the token from the `Authorization: Bearer` header, verifies its signature and expiry, and loads the full user record from the database to populate the `SecurityContext`. Malformed or expired tokens cause the request to proceed as unauthenticated and be rejected by `authorizeHttpRequests`.

CSRF and Stateless Session Management

Because authentication credentials are transmitted via the `Authorization` header rather than a browser-managed cookie, the browser's same-origin policy prevents cross-site requests from including the token, rendering CSRF attacks ineffective. Spring Security's session creation policy is set to `STATELESS`, ensuring no server-side session is created or used.

Cross-Site Scripting (XSS) Prevention

The system defends against Cross-Site Scripting at two independent layers. On the frontend, React's JSX rendering engine HTML-escapes all dynamic values by default before inserting them into the DOM. Any expression of the form `{product.name}` is treated as plain text and never interpreted as executable markup. Critically, the application does not use `dangerouslySetInnerHTML` anywhere in the codebase, so no intentional bypass of this escaping exists. On the backend, every API response carries a `Content-Type: application/json` header, which instructs the browser to parse the body as structured data rather than HTML. File upload endpoints return only a server-generated UUID filename, eliminating the stored XSS vector that arises when user-provided filenames are echoed into the DOM.

Role-Based Authorization

Authorization is enforced at three levels. First, `SecurityConfig.authorizeHttpRequests()` applies coarse-grained URL-pattern rules (e.g., audit logs are ADMIN-only, proposal approval

is MANAGER-only). Second, `@PreAuthorize` annotations on individual controller methods provide fine-grained method-level security (e.g., `@PreAuthorize("hasAnyAuthority('ADMIN', 'MANAGER')")` on product creation). Third, the service layer enforces cross-cutting rules such as preventing non-ADMINs from modifying user roles, ensuring business logic constraints cannot be bypassed by manipulating HTTP requests directly.

Input Validation

Jakarta Bean Validation annotations on request model classes are activated by `@Valid` on controller parameters. Violations are collected by a global handler and returned as a structured HTTP 400 response. On the frontend, Zod schemas paired with React Hook Form provide a matching client-side validation layer with immediate field-level feedback.

File Upload Security

Product image uploads enforce three independent guards: (1) the MIME type must be `image/jpeg`, `image/png`, or `image/webp`; (2) the file extension must match one of `.jpg`, `.jpeg`, `.png`, or `.webp`; (3) the file size must not exceed 5 MB, enforced at the application layer ahead of Spring's `multipart.max-file-size` cap. The saved filename is a UUID with the validated extension, eliminating path traversal and executable-name attack vectors.

Rate Limiting on Login

`AuthController` implements an in-memory sliding-window rate limiter: a `ConcurrentHashMap` maps each client IP to a timestamped request log. If more than 10 login attempts arrive from the same IP within 60 seconds, the endpoint returns `HTTP 429 Too Many Requests`. A scheduled daemon thread purges expired entries hourly. It is acknowledged that this in-memory approach carries a bounded memory risk under high volumes of unique IPs, so a Redis-backed rate limiter is recommended for production.

CORS Configuration

`CorsConfig` restricts cross-origin requests to the value of the `cors.allowed-origins` environment variable, defaulting to `http://localhost:3000` in development and overridden to the production Vercel domain at deployment. All other origins are rejected at the CORS pre-flight stage.

5.3 Error Handling Strategy

Backend (GlobalExceptionHandler)

A single `@ControllerAdvice` class intercepts all exceptions thrown from any controller or service and maps them to structured JSON responses:

Exception	HTTP Status	Trigger
`NotFoundException`	404	Resource ID not found in database
`NameValueRequiredException`	400	Business constraint violated (e.g., insufficient stock)
`InvalidCredentialsException`	401	Wrong username or password
`MethodArgumentNotValidException`	400	Jakarta Bean Validation failure
`ConstraintViolationException`	400	Path or query parameter validation failure
`IllegalArgumentException`	400	Invalid image type or size
`AccessDeniedException`	403	Spring Security role check failed
`UncategorizedSQLException`	500	Low-level JDBC error
`Exception` (catch-all)	500	Any other unhandled exception

All error responses follow the same `Response` JSON envelope, making client-side error handling uniform where the frontend always reads `response.data.message` from the Axios error object.

Frontend (React Query and Axios Interceptors) React Query's `onError` callback on mutations is used to display inline error messages (e.g., "Insufficient stock. Available: 5"). For the CSV export, which cannot use a modal, an inline error state variable in the component replaces the original `alert()` call. The Axios instance in `lib/axios.js` does not include a global error interceptor that redirects on 401, as the `ProtectedRoute` component handles redirection at the route level based on `localStorage` presence.

5.4 Advanced Features Implementation

CSV Export for Nine Entities

A shared utility class `CsvExportUtils` provides `escapeCsv(String)`, which wraps values containing commas, newlines, or double-quotes per RFC 4180, and `isoDateTime(LocalDateTime)` for consistent timestamp formatting. Every export endpoint returns the file with `Content-Type: text/csv; charset=UTF-8` and a `Content-Disposition: attachment` header. Exports are capped at 10,000 rows to prevent out-of-memory conditions on large tables.

A key challenge was ensuring Vietnamese and other non-ASCII characters in product names and supplier addresses rendered correctly when opened in Microsoft Excel. Excel does not automatically detect UTF-8 encoding and will mojibake the file unless a UTF-8 BOM (`0xEF 0xBB 0xBF`) is prepended to the output stream. This was resolved by writing the BOM bytes at the start of the `HttpServletResponse` output stream before any CSV content, which signals the correct encoding to Excel without affecting other consumers such as text editors or data pipelines.

Real-Time Dashboard

The `DashboardService` aggregates several metrics in a single SQL pass against the `transactions` and `products` tables: total active products, total inventory value (sum of `price` × `stock_quantity`), transaction count by type for the current month, and low-stock product count. Performing this aggregation in a single query rather than multiple round-trips avoids N+1 latency on the dashboard load. On the frontend, Recharts renders the metrics as line charts and summary cards.

The main challenge was balancing data freshness against API load, since the dashboard is the first page every user lands on after login. Polling on every mount would hammer the aggregation query under concurrent sessions. This was resolved by configuring React Query with a 60-second `staleTime`, meaning the cached result is reused for subsequent renders within that window and a background refetch is only triggered once the data is considered stale.

Product Image Upload and Serving

Product images are uploaded as `multipart/form-data`. The backend saves each file to the `uploads/products/` directory under a UUID filename and stores the relative path in the database. `UploadResourceConfig` maps the `/uploads/**` URL pattern to this filesystem directory so images are served directly as static resources.

A deliberate trade-off was made in configuring this URL pattern as `.permitAll()` in `SecurityConfig`, bypassing the Spring Security filter chain for image requests. This was necessary because image URLs are embedded directly in `<img>` tags, which cannot attach an `Authorization` header. The risk is accepted given that product images contain no sensitive information. A known limitation of this approach is that the `uploads/` directory is not persisted across Railway deployments; a production-grade solution would use object storage such as AWS S3 or Cloudflare R2 instead.

Audit Log System

Every significant state mutation — product creation and update, transaction logging, proposal approval and rejection, and password changes — writes a record to the `audit_logs` table via `AuditLogService.logAction()`. Each record captures the `user_id` of the actor, an `action`

string, the `entity_type` and `entity_id` of the affected object, a human-readable `details` field, and a `created_at` timestamp. The audit log UI is read-only and restricted to ADMIN users.

The primary challenge was ensuring consistent coverage across all mutation paths without duplicating logging logic. Since each service method calls `AuditLogService.logAction()` explicitly, there is a risk that a newly added endpoint omits the call. This was mitigated by treating audit logging as a mandatory step in the service layer code review checklist and verifying coverage manually via Postman during testing.

SearchableSelect Component

Standard HTML `<select>` elements become unusable when a dropdown contains hundreds of products or suppliers, as there is no native mechanism to filter options by typing. A custom `SearchableSelect.jsx` combobox component was built using React state and `useRef`, supporting keyboard navigation and a `renderOption` prop to customise how each option is displayed (e.g., showing both product name and SKU together). It integrates with React Hook Form via the `Controller` wrapper and is used on the Purchase, Sale, and Return transaction pages.

The main implementation challenges were click-outside dismissal and React Hook Form integration. Click-outside detection required attaching a `mousedown` listener to `document` via `useEffect` and comparing the event target against the component's `useRef` reference to determine whether the click originated inside or outside the dropdown. Integrating with React Hook Form required using the `Controller` wrapper to bridge the library's `onChange`/`value` interface with the component's internal state, since React Hook Form cannot directly control uncontrolled custom components.

5.5 AI Assistance Report

AI coding assistants (Claude and Gemini Code Review) were used throughout the project as a pair-programming tool. The following describes three representative examples.

Example 1: AI as the Project Manager and Quality Assurance

In order that the team misses no requirements, a customized Claude Code skill (`/compare`) was created. This skill is designed and trained over multiple iterations to catch any deviation from the requirement list and propose a workflow for the necessary features. Furthermore, this skill acts as a checklist and issues an audit report to verify if any requirements are not met. Then, Gemini Code Review embedded onto GitHub catches several remaining issues before the pull request is accepted and merged into the system.

Example 2: SSE Notification Stream

Neither team member had prior experience with Server-Sent Events in Spring Boot. Thus, Claude was prompted for a basic SSE emitter pattern but the generated code did not handle thread safety. The issues are that `List<SseEmitter>` was an `ArrayList` (not thread-safe) and the `emitter.send()` call was not synchronized. Both issues were identified during code review and corrected: `ArrayList` was replaced with `CopyOnWriteArrayList` for safe concurrent iteration, and all `emitter.send()` calls were wrapped in `synchronized(emitter)` blocks. In this solution, the scheduler and cleanup logic were written entirely without AI assistance.

Example 3: Zod Schema Composition on the Frontend

Zod schemas for the Purchase and Sell forms were initially written with duplicated field definitions. Claude suggested extracting common fields into a shared base schema and using `.extend()` to create per-form variants. This solution was adopted, reducing schema boilerplate significantly.

What We Learned

AI assistance is most productive when the developer understands the domain well enough to evaluate the output. In both Java examples above, the generated code required security or concurrency fixes that only became apparent through code review. The assistants were excellent

at eliminating boilerplate but less reliable for subtle correctness concerns. The team adopted the practice of never committing AI-generated code without running the relevant tests.

6 Testing and Quality Assurance

6.1 Testing Strategy

Our testing strategy prioritized ensuring data integrity, system security, and a seamless user experience across the full stack. The testing was conducted manually using a Black-Box approach, focusing on end-to-end business workflows, API response validation, and UI/UX responsiveness.

We divided our testing into several domains:

- **Authentication and RBAC Testing:** We utilized three dedicated test accounts (admin1@gmail.com, manager1@gmail.com, and staff1@gmail.com) to systematically verify that Role-Based Access Control (RBAC) correctly restricted frontend routes and backend API endpoints.
- **Workflow Integration Testing:** We traced the complete lifecycle of core business operations, such as creating a product, logging a purchase to increase stock, logging a sale to decrease stock, and processing a return.
- **Validation and Security Testing:** We aggressively tested edge cases, including oversized file uploads, invalid data types, and attempting to force negative stock levels, to ensure the backend responded with proper HTTP error codes rather than application crashes.
- **UI/UX Testing:** The frontend was tested using browser developer tools to simulate various screen resolutions (Desktop 1920x1080, Tablet 768x1024, Mobile 375x667) ensuring Tailwind CSS media queries rendered correctly.

Test Scenarios

1. **Test user registration with duplicate email:** Attempted to register a new user using an email already existing in the database.

   Output: System rejects input and displays a clear "Email already in use" error message.

2. **Test login with invalid credentials:** Attempted login with an incorrect password.

   Output: Rejected with a 401 Unauthorized status and appropriate UI error banner.

3. **Test Role-Based Access Control (RBAC) enforcement:** Logged in as a STAFF user and attempted to navigate to the Admin User Management dashboard via URL.

   Output: User is blocked and redirected to an unauthorized/home page.

4. **Test oversized image upload:** Attempted to upload a product thumbnail exceeding the 5MB limit.

   Output: Frontend validation catches the file size before submission; backend catches it if bypassed, returning a 400 Bad Request.

5. **Test product filtering and pagination:** Navigated to the product list and applied filters for "Out of Stock" while setting page size to 10.

   Output: Data table correctly updates via server-side pagination without losing the filter state.

6. **Test transaction stock increment (Purchase):** Created a "Purchase" transaction for a product.

   Output: Product's stock quantity immediately and automatically increases by the purchased amount.

7. **Test transaction stock decrement prevention (Overselling):** Attempted to record a "Sale" transaction for 50 units of a product that only has 10 units in stock.

   Output: Form submission is blocked, preventing negative stock values.

8. **Test soft-deletion integrity:** Admin deletes a Category that is currently linked to existing products.

   Output: The category is marked as INACTIVE (soft-deleted) rather than permanently dropped, preserving historical product data.

9. **Test user password change:** Logged in and successfully changed the user password, then attempted to log out and log back in with the old password.

   Output: Old password fails; new password grants access.

10. **Test CSV export functionality:** Clicked the export button on the Transactions page.

    Output: System successfully generates and downloads a properly formatted CSV file containing the filtered list of transactions.

11. **Test real-time validation on forms:** Typed invalid characters into the "Price" field of the product creation form.

    Output: Zod/React Hook Form immediately highlights the field in red before the user clicks submit.

12. **Test dashboard data aggregation:** Logged a mix of sales and purchases, then navigated to the Dashboard.

    Output: Line charts and summary statistics (Total Value, Total Transactions) correctly reflect the newly added data.

6.2 Bug Tracking

During the development lifecycle, the team encountered and resolved several complex logical and state-management bugs to ensure system stability:

1. **Infinite Redirect Loop on Authentication**

   - **Issue:** When an unauthenticated user attempted to access a protected URL directly, the system correctly redirected them to the Login page and saved their intended destination. However, upon successful login, a flaw in the React Router state management caused the app to rapidly loop between checking the authorization token and attempting to load the previous page, crashing the browser tab.
   - **Fix:** We resolved this by refactoring the useEffect dependency array in our authentication context. We ensured that the "redirect path" state was explicitly cleared from memory immediately after the first successful navigation post-login, breaking the infinite cycle.

2. **Double Stock Deduction on Return-to-Supplier Approval**

- **Issue:** When a Manager approved a Return-to-Supplier proposal, the stock quantity of the linked product was being decremented twice: once immediately inside `returnToSupplier()` when the transaction was first created as `COMPLETED`, and a second time inside `updateTransactionStatus()` when the proposal approval loop attempted to mark the linked transaction as `COMPLETED` again. This caused products to lose twice the intended quantity.
- **Fix:** The redundant stock-decrement block inside `updateTransactionStatus()` was removed for the `RETURN_TO_SUPPLIER` branch, since return transactions are now created in a single step already in the `COMPLETED` state. `ProposalServiceImpl.executeProposal()` was also updated to set the `RETURN_TO_SUPPLIER` proposal result to `COMPLETED` directly, ensuring exactly one stock mutation per return event.

3. **Desynchronized Pagination with Dynamic Filtering**

   - **Issue:** When viewing the Transactions table, applying a specific filter (viewing only "Returns") correctly filtered the data down to 6 items. However, the pagination component still displayed the total for the unfiltered database (21 items) and allowed the user to click to "Page 2", which resulted in an empty, broken table since page size was set to 10.
   - **Fix:** This required a two-part fix. First, on the backend, we ensured the Spring Boot API returned the filtered totalElements count rather than the absolute table count. Second, on the frontend, we added a trigger in React that automatically resets the currentPage state back to 1 (or 0 for the backend API) whenever any filter criteria changes.

4. **Client-Side Route Protection Failure**

   - **Issue:** Initially, some protected frontend views were not properly wrapped in our authorization logic. While the backend API correctly blocked unauthorized data requests (returning 403 Forbidden), an unauthorized user could still technically load the URL and see the "empty" shell of manager-level pages before the data fetch failed.
   - **Fix:** We implemented a strict ProtectedRoute higher-order component (HOC) in App.js. This component evaluates the user's decoded JWT role before attempting to render the route's children. If the role does not match, the user is immediately redirected to an "Unauthorized" page, preventing any secure UI elements from flashing on the screen.

6.3 Known Limitations

While the Inventory Management System successfully fulfills its core requirements, there are a few practical limitations in its current iteration:

1. **Local File Storage for Images:** Currently, uploaded product images are saved directly to a local /uploads directory on the backend server.

   - **Reason:** This was done to simplify the local development environment for the scope of a course.
   - **Future Plan:** If deployed to a cloud server (which often uses ephemeral file systems that wipe local files on restart), this approach will cause data loss. The future plan is to migrate image hosting to a dedicated cloud storage bucket, such as AWS S3 or Cloudinary.

2. **Manual Data Entry for Operations:** Warehouse staff currently have to manually search for and select products using text input when creating a transaction.

- **Reason:** Integrating hardware peripherals was outside the scope of the software-focused curriculum.
- **Future Plan:** Integrate a React-based barcode or QR code scanning library utilizing device cameras, allowing staff to rapidly process physical items on the warehouse floor using a tablet or mobile device.

3. **Delayed Role Revocation Due to Stateless JWTs:** If an Administrator updates a user's role (e.g., demoting a Manager to Staff) or disables an account, that user can theoretically continue to perform Manager actions until their current JSON Web Token (JWT) expires. The current token lifetime is set to six months, a pragmatic simplification for a course project that avoids login fatigue during development and evaluation but is unsuitable for production.

   - **Reason:** The system uses standard stateless JWTs for authentication, which are validated by checking the signature rather than querying the database on every request. This is great for performance but means tokens cannot be instantly revoked.
   - **Future Plan:** Implement a Redis-based token blocklist (denylist) to store invalidated tokens, or transition to a short-lived Access Token (15 minutes) + long-lived Refresh Token architecture to minimize the vulnerability window.

4. **Weak Demo Passwords:** All three seeded test accounts use the password `12345678`.

   - **Reason:** Simple credentials are intentional for evaluator convenience, allowing graders to log in quickly without managing a secrets file.
   - **Production note:** In any real deployment these would be replaced with strong, randomly generated credentials and distributed securely.

7 Deployment

7.1 Deployment Process

The system uses a split-deployment strategy: the React frontend is deployed to Vercel's global CDN, while the Spring Boot backend and MySQL database are hosted on Railway. A `docker-compose.yml` is also provided at the repository root for local full-stack development.

Frontend Deployment (Vercel)

1. The `frontend/` directory is connected to a Vercel project via the Vercel GitHub integration.
2. A single environment variable is set in the Vercel dashboard via `REACT_APP_API_BASE_URL`
3. Every push to the `master` branch triggers an automatic Vercel build (`npm run build`) and deployment.
4. Vercel serves the static build output from a global CDN with automatic HTTPS.

Backend and Database Deployment (Docker Compose)

The `docker-compose.yml` at the repository root defines three services:

1. **db** (`mysql:8.0`): Starts a MySQL 8 container, auto-creates the database, and runs `inventory_db.sql` as an initialization script on first boot. A named volume `ims-db-data` persists data across container restarts. A `healthcheck` using `mysqladmin ping` ensures the backend service only starts after MySQL is ready.

2. **backend** (built from `Backend/Dockerfile`): Runs the Spring Boot JAR. All sensitive configuration is supplied via environment variables defined in a `.env` file at the repository root (not committed to Git):

3. **frontend** (built from `frontend/Dockerfile`): Builds the React app and serves it with Nginx on port 80. The `REACT_APP_API_BASE_URL` build arg is passed in from `.env`.

Full-Stack Local Setup (Step-by-Step)

1. Clone the repository: `git clone https://github.com/.../inventory-management.git`
2. Copy `.env.example` to `.env` and fill in credentials.
3. Run: `docker compose up --build`
4. Wait for the `ims-backend` container to print `Started BackendApplication`. MySQL initialization may take 10–15 seconds on the first run.
5. Import the seed data: `docker exec -i ims-mysql mysql -u root -p${MYSQL_ROOT_PASSWORD} inventory_db < database/inventory_db_data.sql`
6. Access the application at `http://localhost:3000` (frontend).

Manual Backend-Only Setup (without Docker)

1. Ensure Java 21 and MySQL 8 are installed.
2. Create the schema: `mysql -u root -p < database/inventory_db.sql`
3. Seed data: `mysql -u root -p inventory_db < database/inventory_db_data.sql`
4. Copy `Backend/src/main/resources/application.properties.example` to configure your own credentials.
5. From the `Backend/` directory: `.\mvnw.cmd spring-boot:run` (Windows) or `./mvnw spring-boot:run` (Unix).
6. From the `frontend/` directory: copy `.env.example` to `.env`, then `npm install && npm start`.

7.2 CI/CD Pipeline

Evolution from VPS to Cloud Platforms

During the initial VPS deployment phase, a GitHub Actions workflow (`.github/workflows/deploy.yml`) provided a full automated pipeline for both the frontend and backend. On every push to `master`, the workflow executed the following stages:

1. Built the React frontend via `npm ci && npm run build`
2. Built the Spring Boot backend via `./mvnw clean package -DskipTests`
3. Copied the compiled artifacts to the VPS over SCP
4. SSH'd into the VPS and restarted the `systemd` service to apply the new deployment

VPS credentials (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`) were stored as GitHub Secrets and never committed to the repository.

Current Pipeline (Vercel + Railway)

Following the migration from the VPS to a split cloud deployment, the GitHub Actions workflow was retired. Both platforms provide native GitHub integration that absorbed its responsibilities:

- **Vercel** triggers an automated build and production deployment on every push to `master`, and generates isolated Preview deployments for pull request branches, allowing UI changes to be reviewed at a unique URL before merging.
- **Railway** triggers an automated redeploy of the Spring Boot backend on every push to `master` via its GitHub integration, building and containerising the application without manual intervention.

Current Limitation and Future Improvement

The current pipeline lacks a formal CI validation stage for the backend. No GitHub Actions workflow currently runs the test suite or enforces a successful build before Railway's deployment step is triggered. A malformed commit could therefore be deployed directly to production. Thus, establishing a GitHub Actions workflow to run `./mvnw clean verify` on every push is identified as the primary CI/CD improvement for a future iteration.

7.3 Monitoring and Maintenance

Application Logging

The backend uses the Logback logging framework (configured in `logback-spring.xml`), which is the default for Spring Boot. Key events are logged at different levels:

- **INFO**: Successful JWT validation, application startup events.
- **WARN**: Could not resolve the current user ID for audit (non-critical).
- **ERROR**: JWT validation failures, unhandled exceptions caught by `GlobalExceptionHandler`, SSE emitter removal on disconnect.

In production, log output can be redirected to a file or aggregated by a log management service (e.g., Logtail, Datadog) by setting the appropriate Logback appender.

In-Application Audit Trail

The `audit_logs` table serves as an immutable application-level audit trail. Every create, update, delete, and transaction event writes a record with the acting user's ID, the action name, the affected entity type and ID, a plain-text detail message, and a timestamp. An Administrator can query and export these logs from the Audit Logs page to investigate data anomalies or security incidents without requiring direct database access.

Database Backup Strategy

For the Docker Compose deployment, the MySQL data volume (`ims-db-data`) stores all data. A simple backup can be taken at any time with:

```
docker exec ims-mysql mysqldump -u root -p${MYSQL_ROOT_PASSWORD} \
  inventory_db > backup_$(date +%Y%m%d).sql
```

Automated daily backups using a cron job or a Docker sidecar container (e.g., `fradelg/mysql-cron-backup`) are recommended for any production deployment.

8 User Guide

8.1 Getting Started

The system can be accessed via https://inventory-management-sigma-puce.vercel.app with the following credentials:

Role	Email Address	Password	Primary Capabilities
ADMIN	admin1@gmail.com	12345678	User Management, Audit Logs and all existing data.
MANAGER	manager1@gmail.com	12345678	Access to Dashboard, data for Products, Categories, Suppliers, Warehouses, Proposals and all Transaction types (Purchase, Sell, Return).
STAFF	staff1@gmail.com	12345678	Perform Transactions.

Although the page implements a registration form, it should be noted that all accounts are created by administrators to be used internally. The form, nonetheless, only allows for STAFF accounts which have no access to modify data without awareness from accounts with higher roles.

8.2 Main Feature Walkthroughs

1. **Global Features**

   Dashboard Analytics:

   - Upon accessing the dashboard, the user will see six primary summary cards at the top of the screen. These provide an immediate snapshot of inventory health (Total Products, Low Stock, Out of Stock) and financial standing (Retail Stock Value, Inventory Cost, and Estimated Margin).
   - **Data Export:** Click the **"Export CSV"** button in the top right corner to download a spreadsheet snapshot.

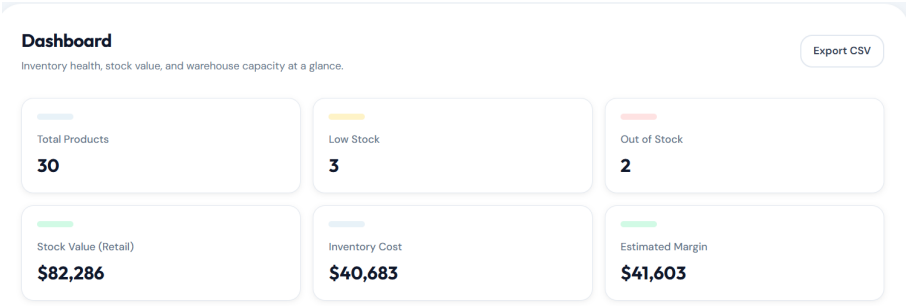


Figure 4: Dashboard Summary

   - **Net Daily Sales Tracking:** Scroll to the Net Daily Sales line chart to monitor daily revenue. Use the dropdown menus to filter the data by specific months and years.



Figure 5: Net Daily Sales Chart

- **Warehouse Utilization:** Visualize how stock is distributed across different physical locations (e.g., comparing capacity between HCMC Main, Hanoi North, and Da Nang Central).

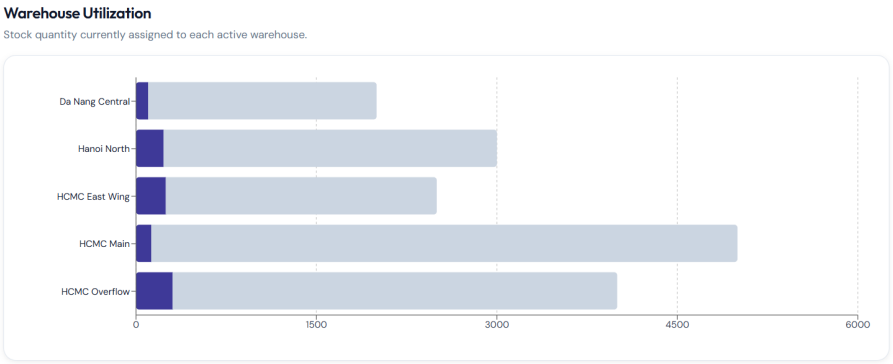


Figure 6: Warehouse Utilization Chart

2. **User & System Administration**

   Access Level: `[ADMIN ONLY]`

   User Management:

   - **Navigation & Overview:** Click the **"Users"** tab in the left-hand sidebar to access the centralized account management dashboard.
   - **Search & Filter:** Quickly locate specific employees by typing their name or email into the search bar. You can further narrow down the table using the **Role** and **Status** dropdowns, or by selecting a specific date range using the **"Joined from / to"** calendar inputs.
   - **Inline Management:** Update an employee's access level using the Role dropdown or immediately revoke their access by clicking the red **"Deactivate"** button.
   - **Data Export:** Click the **"Export CSV"** to download a current snapshot.

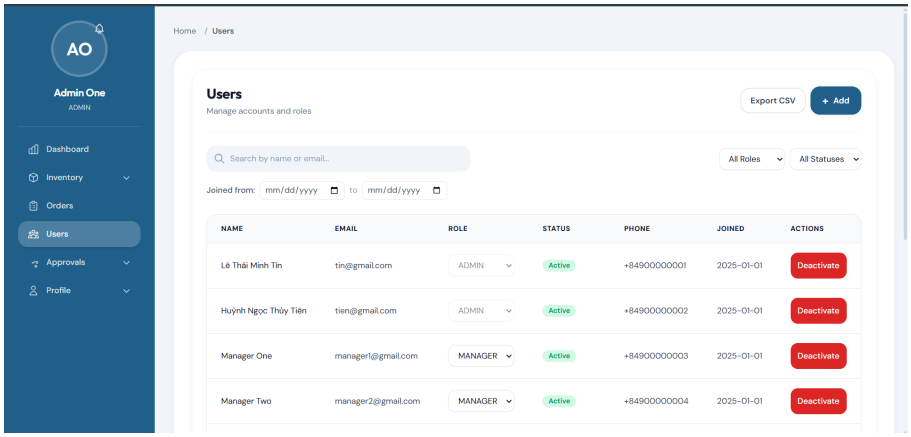


Figure 7: User Management Dashboard with Search and Filter Controls

Creating a New User:

- Click the **"+ Add"** button at the top right of the Users dashboard.
- Fill in the employee's designated details: User Name, Email, Phone Number, and a secure temporary Password.
- Assign the appropriate system **Role** (ADMIN, MANAGER, or STAFF) from the dropdown selector.
- Click **"Add Create"** to officially provision the account and grant them access to the system.

Audit Logs:

- **Navigation & Filtering:** Navigate to **Approvals** and drop down to see the **Audit Logs**. Filter the logs using the **All Actions** dropdown (PROPOSAL CREATED, TRANSACTION EXECUTED, RETURN TO SUPPLIER) or by specific entity types and date ranges.
- **Check Details:** Click the **"Details"** button on any log entry to view the exact granular details of the event.

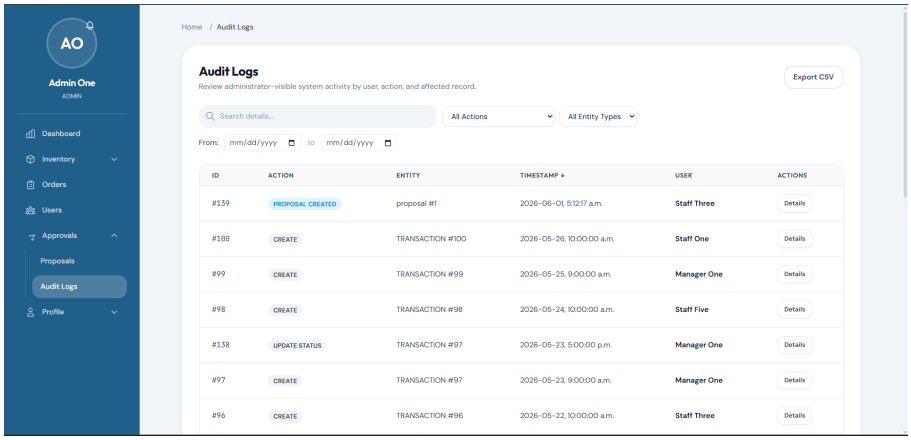


Figure 8: Audit Logs Dashboard showing system-wide tracking

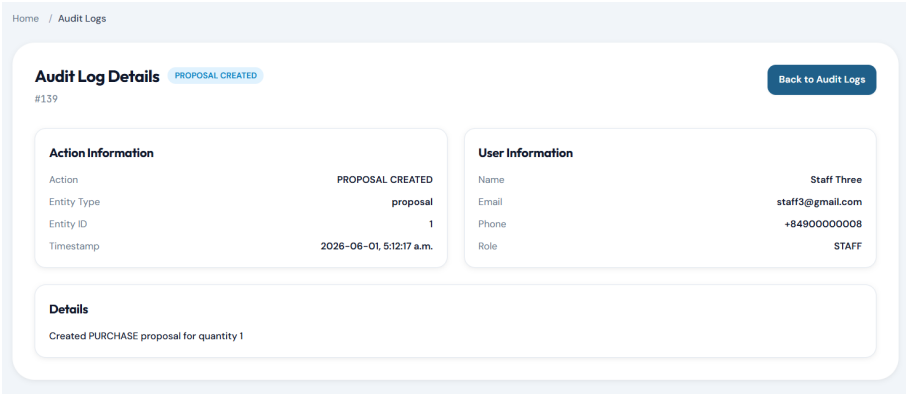


Figure 9: Audit Log Details revealing specific granular data for a user action

3. **Proposal Pipeline:**

   - **Use-case:** Transactions initiated by Staff members are recorded as "Pending Proposals" that require higher-level review before writing to the database.
   - **Reviewing Pending Actions:** As a Manager, navigate to **Approvals**, drop down to **Proposals** to view the queue of pending inventory actions.
   - **Verification Process:** On the details screen, you can cross-reference all critical data simultaneously before approving.

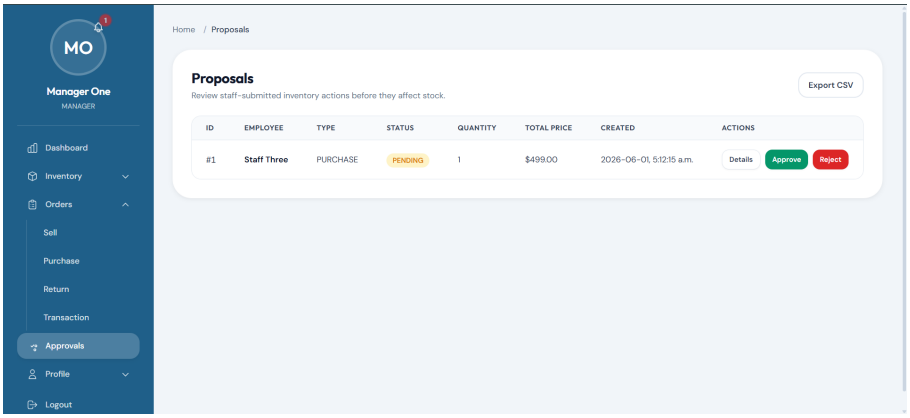


Figure 10: Proposals Queue awaiting manager's action

4. **Master Data & Inventory Control**

   Access Level: `[ADMIN, MANAGER]`

   Product Catalog Dashboard:

   - **Navigation & Overview:** Navigate to **Inventory > Product** to view the complete catalog.
   - **Search & Advanced Filtering:** Quickly locate items using the search bar with supported filters.
   - **Data Export:** Click the **"Export CSV"** button to generate a current snapshot.

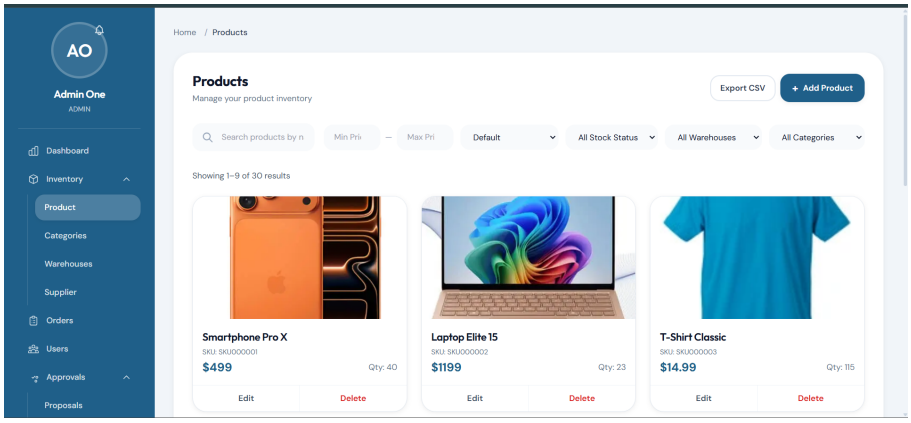


Figure 11: Product Catalog Dashboard with advanced search and filtering

Creating a New Product:

- Click the **"+ Add Product"** button to open the creation form.
- Input the core product details: Product Name, Description, Selling Price, Initial Stock Quantity, and the Reorder Threshold (the baseline number that triggers low-stock warnings).
- Upload a representative Product Image using the file browser preview.
- Link the product to existing Master Data by selecting the appropriate **Category** and **Warehouse** from the dropdown menus.
- Click **"Create Product"**. The system will securely save the item and automatically generate a unique SKU.

Editing Products & Managing Linked Suppliers:

- From the Product Dashboard, click **"Edit"** on any item card to modify its baseline details.
- **Supplier Management:** Scroll down to the **Suppliers** section to link multiple approved vendors to the specific product.
- **Unit Costs & Defaults:** For each linked supplier, you can define their specific Purchase Cost / Unit. If multiple suppliers provide the same item, you can click **"Use as default"** to designate the preferred vendor, which streamlines the auto-fill process for future purchase proposals.

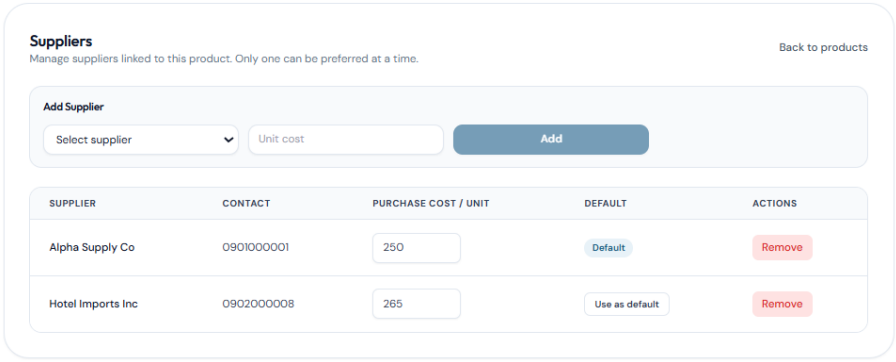


Figure 12: Supplier Management panel within the Product Edit view for defining default vendors and unit costs

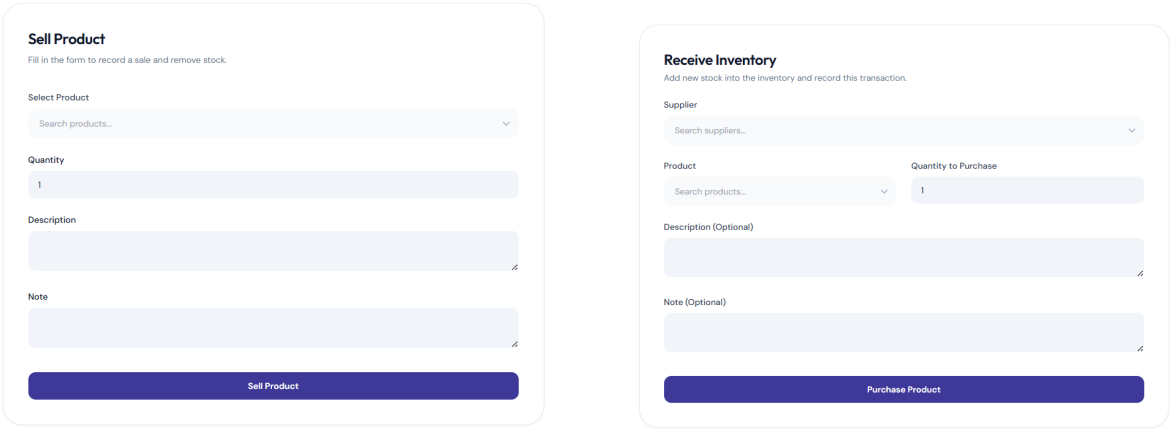
5. **Order Processing**

   Access Level: `[MANAGER, STAFF]`

   **(Note: Administrators are restricted to read-only views of the transaction logs to maintain separation of operational duties.)*

   Processing Sales or Purchases:

   - **Access:** `[MANAGER, STAFF]`
   - Navigate to **Orders** and select the appropriate form to fill.
   - When a user selects a product from the dropdown, the system displays the current available stock.



Sell Product form Purchase Product form

Figure 13: Transaction forms for inventory operations

   Processing Returns (Deducting Stock to Supplier):

   - **Access:** `[MANAGER ONLY]`
   - Navigate to **Orders > Return** to process defective or excess inventory being sent back to a vendor. The form is similar to Sell and Purchase.
   - **Dynamic Relational Filtering:** To ensure data accuracy, the manager must select the Product *first*. Once a product is chosen, the system dynamically filters the Supplier dropdown to only display the specific providers.

6. **System Notifications & Alerts**

   Access Level: `[ADMIN, MANAGER]`

   Role-Aware Alert Delivery:

   - The notification engine is deeply integrated with the Role-Based Access Control where alerts are only dispatched to relevant users.
   - A notification bell icon is permanently pinned to the user profile badge in the navigation menu with a red indicator that displays the current count of unread alerts.
   - Clicking the bell opens a quick-view, or click **"View All Notifications"** to open the full dashboard.

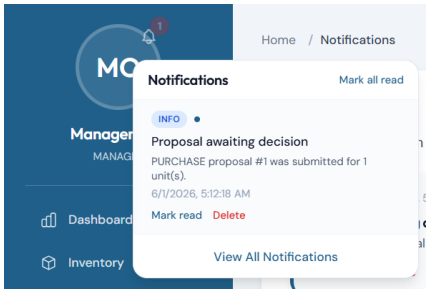


Figure 14: Quick-access notification bell modal providing immediate workflow updates

9 Team Collaboration

9.1 Team Organization

- **Division of Labor:** The project was first divided using a standard frontend and backend split. The project manager then reviewed the system and proposed features to meet the requirements. Finally, extensive testing is done to ensure the website's usability.
- **Meeting Schedule:** The team conducted weekly meetings to catch up and discuss features to be implemented.
- **Communication Tools:** Messenger or face-to-face meetings are mainly used to drive the communication.

9.2 Individual Contributions

- **Lê Thái Minh Tín: (*Git: Dorecepha*):**
  - *Database Engineering:* Designed a normalized domain model (3NF) containing 12 normalized tables and implemented strict referential integrity, cascading updates/deletes, as well as handling multiple one-to-many and many-to-many relationships via junction tables.
  - *Business Logic & Integration:* Developed the core CRUD APIs, linked frontend routes to backend controllers, and implemented complex inventory algorithms (stock tracking, automatic reorder thresholds, and overselling prevention).
  - *Security Implementation:* Enforced strict non-negotiable security measures, including BCrypt password hashing, JWT authentication, role-based authorization, SQL injection prevention via parameterized queries, CORS/CSP headers, and rate limiting.
  - *Project Manager & Quality Assurance*: Constantly audited the system to spot inefficient logic, security loopholes and proposed features and UI designs to address the requirement list sufficiently.
  - *Lines of Code:* 60,064 added, 10,416 removed (Net total: 49,648 lines).
- **Huỳnh Ngọc Thủy Tiên (*Git: Tn-H4*)**
  - *Interface Development:* Engineered the user-facing application, building comprehensive views, data lists, and an interactive notification bell system.
  - *Form & Schema Management:* Developed robust `schema.js` architectures to handle complex data entry for the Create and Edit modules.

- *Responsive UI/UX & QA:* Ensured design consistency across the entire application interface, adapting layouts for desktop, tablet, and mobile viewing. Conducted rigorous QA testing to ensure seamless user interaction.
- *Lines of Code:* 7,741 added, 3,278 removed (Net total: 4,463 lines).

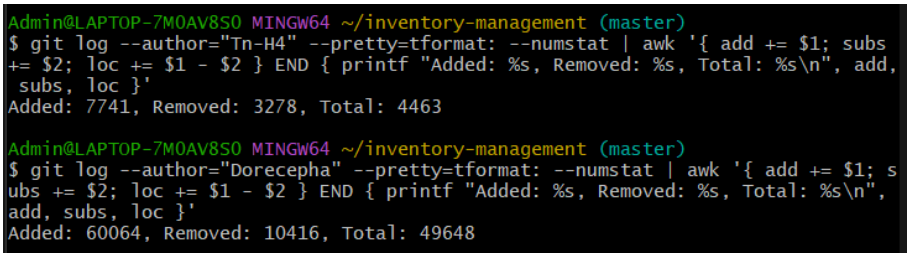


Figure 15: Git terminal output verifying individual line contributions for both team members.

9.3 Challenges in Teamwork

- **Systematic Bug Fixing (Pagination):**
  - *Issue:* Pagination components failed to render data correctly on several list views.
  - *Resolution:* This required a collaborative full-stack debugging session. Once the root cause was identified across both the frontend state and backend query limits, we systematically applied the patch to all relevant pages to prevent future technical debt.
- **Deployment Infrastructure Constraints:**
  - *Issue:* During the initial public hosting phase, the application experienced long load times and eventual crashes due to heavy server-side processing demands.
  - *Resolution:* We audited our deployment strategy and migrated our frontend hosting to Vercel. This optimized our delivery network, handled the load efficiently, and restored system stability.

10 Conclusion and Reflection

10.1 Project Summary

The final iteration of the Inventory Management System successfully met and exceeded the initial baseline requirements set forth in the project rubric, achieving a highly scalable and secure application architecture. The initial plan called for a standard CRUD application with 6-8 entities; the final product delivered a robust, normalized 3NF database containing 12 tables.

Furthermore, the team successfully implemented all three targeted advanced features: comprehensive Data Management, a role-aware System Notification engine and an Analytics Dashboard leveraging Recharts for data visualization. The project maintained strict separation of concerns, culminating in a highly responsive, secure, and production-ready application.

10.2 Lessons Learned

- **Raw JDBC forces genuine SQL fluency.** Deliberately avoiding an ORM meant every query, join, and constraint had to be written and reasoned about, producing a far deeper understanding of referential integrity and index design.

- **Stateless JWT authentication eliminates an entire class of vulnerabilities.** Bearer-token authentication via the `Authorization` header naturally removes the session state that CSRF attacks depend on, demonstrating that the right architectural choice can resolve a security concern without additional countermeasures.
- **Deployment environment matters as much as code quality.** Hosting the frontend on a general-purpose VPS introduced load-time penalties that disappeared once it was migrated to Vercel's CDN. This reinforced that a decoupled architecture requires equally deliberate infrastructure decisions for each layer.
- **Proactive transparency about trade-offs is a legitimate discipline.** Acknowledging known limitations (disabled frontend auth guards, long-lived JWT expiry, no refresh token) and documenting the justification for each enforced a clearer understanding of what was deliberately deferred.

10.3 Individual Reflections

Reflection: Lê Thái Minh Tín

This project was my first time designing and building a production-grade system end-to-end rather than following a textbook exercise. The most consequential architectural decision was deliberately bypassing JPA and writing every SQL statement by hand with raw JDBC. This was difficult at first but it eventually culminated in a deeper understanding of query planning and index design that I would not have acquired otherwise. Additionally, seeing query performance improve measurably after adding composite indexes made the relationship between schema design and runtime behaviour tangible.

The security layer was the most intellectually rewarding part of the project. Implementing JWT from the ground up gave me a concrete understanding of stateless authentication. A key insight was that stateless JWT delivered via the `Authorization` header is inherently immune to CSRF, because there is no browser-managed cookie for a malicious page to exploit. Understanding this removed what had been a confusing abstraction.

The proposal approval workflow was the most complex business logic I designed. Ensuring that a STAFF-initiated purchase atomically creates both a `PENDING` proposal and a linked `PENDING` transaction, and that a Manager approval increments stock exactly once while simultaneously sending a notification to the original proposer, required coordinating four services in the correct sequence. The double-deduction bug where returns decremented stock twice taught me how critical it is to define a single, authoritative place for each side effect rather than allowing the same action to be triggered from multiple entry points.

If I were to start the project over, I would define API contracts and Swagger documentation before writing any controller code, because early in development, the frontend was building against endpoint shapes that were still changing on the backend, creating repeated integration friction. Thus, agreeing on the `Response` envelope structure and documenting each endpoint upfront would have eliminated that bottleneck entirely. In the future, I intend to explore event-driven architectures and message queues for systems where notification delivery guarantees matter more than the best-effort SSE model implemented here.

Reflection: Huỳnh Ngọc Thủy Tiên

Building the front-end of this application was a huge leap in managing complex client side state. Moving from older desktop architectures such as Java Swing to a fully decoupled, modern React 19 environment required a whole paradigm shift in how I dealt with data flow. I designed the core user-facing architecture and implemented strict role-based route guards such as the

Admin-only and Manager-only layers to ensure the UI perfectly reflected the backend's security permissions, without exposing restricted layouts.

As the primary frontend developer and tester, I focused heavily on making the application both resilient and user-friendly. Using React Hook Form and Zod schemas, I was able to build very robust Create and Edit modules. Having to write these complex validation schemas meant I could catch user errors such as missing required fields, or submitting invalid stock quantities, before they even hit Tín's backend API, greatly improving the user experience.

In my manual QA and testing, I was relentless in finding edge cases for desktop, tablet, and mobile breakpoints. I've also fixed important bugs in the UI, like global pagination limits that were not showing the data properly. I've caught sync issues when wrong API links were generating blank spaces in the UI etc. I had to debug full-stack and communicate much better with the backend to make sure our deployment timelines met.

A big part of my growth came from understanding the bridge between the client and server. Understanding the fundamentals of computer networks made it much easier for us to debug our Vercel migration and CORS setups as we moved into the API integration and deployment stages. I also developed a deep appreciation for accessibility (a11y) and scalable code design. Writing our own reusable hooks cuts down on dozens of lines of duplicated code across our nine data tables. Also, ensuring all the interactive elements, from the notification bell to the transaction forms, had the correct ARIA labels made the application truly professional. In the future, I want to implement more optimistic UI updates so that the transaction processing feels even more instantaneous to the end user.

10.4 Future Enhancements

Given more time and resources, the following enhancements would meaningfully extend the system's capabilities:

- **Automated Reorder Workflows and Supplier Integration:** The current system flags products whose stock falls below a configurable threshold but requires a manager to act manually. A logical next step is a fully automated purchase-order pipeline, where a triggered reorder flag would make the system generate a draft purchase order, route it for manager approval and dispatch it to the supplier via an external API or email template. This would represent the system's most impactful business logic upgrade.
- **WebSocket-Based Real-Time Collaboration:** The existing notification engine uses Server-Sent Events (SSE), which is a unidirectional server-to-client push channel. Upgrading to WebSocket would enable bidirectional communication, supporting use cases that SSE cannot: for example, live stock-level updates visible to all warehouse operators simultaneously during a receiving session, or instant manager-to-staff messaging without a polling round-trip.
- **Caching and Query Optimisation for the Analytics Dashboard:** Analytics queries aggregate transaction data across the full history of the database, making them the most computationally expensive operations in the system. Introducing a caching layer such as Redis would significantly reduce database load during peak reporting hours and allow the system to scale.
- **Offline-Capable Mobile Interface:** Warehouse operators frequently work in environments with unreliable network connectivity. A Progressive Web App (PWA) or dedicated mobile client with a local-first data store (e.g., IndexedDB) would allow staff to log stock movements offline, with changes synchronised to the backend once connectivity is restored. This would also naturally motivate the formal `/api/v1/` versioning strategy, since a mobile client would need a stable, versioned contract independent of the web frontend's release cycle.

Appendices

Appendix A: Meeting Logs

The team conducted five structured meetings over the five-week development period. Day-to-day coordination was handled asynchronously via Messenger; the sessions below represent formal synchronisation points at which decisions with architectural or scheduling impact were made and recorded.

#	Date	Format	Agenda	Outcome
1	Week 1	Face-to-face	Project scope definition; entity identification; technology stack decision	Agreed on Spring Boot + React + MySQL; defined 12-table domain model; assigned frontend/backend responsibilities
2	Week 2	Messenger	Database schema review; API contract draft	Finalised ERD; agreed on unified `Response` envelope structure; documented endpoint shapes before implementation began
3	Week 3	Face-to-face	Mid-project integration review; pagination bug triage	Identified root cause of pagination desync across frontend state and backend query limits; fix assigned and applied to all affected pages
4	Week 4	Messenger	Deployment strategy review; VPS-to-Railway migration decision	Agreed to migrate backend and database to Railway; Vercel confirmed for frontend CDN; CORS allowed-origins updated via environment variable
5	Week 5	Face-to-face	Final QA pass; report review; demo rehearsal	Confirmed all 20 user stories implemented; report sections cross-reviewed; demo flow rehearsed with pre-seeded test data

Table 1: Team meeting log across the five-week project timeline.

Appendix B: API Documentation

All endpoints are prefixed with `/api`. A live interactive version is available at https://inventory-management-production-6424.up.railway.app/swagger-ui/index.html. All authenticated requests require the header `Authorization: Bearer {token}`.

Authentication (`/api/auth`)

Method	Path	Auth	Role	Description
POST	/api/auth/login	None	--	Authenticate; returns JWT
POST	/api/auth/register	None	--	Self-register as STAFF

Login request body: `{"email": "admin@gmail.com", "password": "123456"}`
Login response:
`{"status":200,"message":"Login Successful","token":"eyJ...","role":"ADMIN"}`

Users (/api/users)

Method	Path	Auth	Role	Description
GET	/api/users/all	Yes	ADMIN	Paginated user list
GET	/api/users/{id}	Yes	Any	Get user by ID
PUT	/api/users/update/{id}	Yes	Any	Update user profile
DELETE	/api/users/delete/{id}	Yes	ADMIN	Soft-delete user
PUT	/api/users/activate/{id}	Yes	ADMIN	Reactivate user
GET	/api/users/current	Yes	Any	Current logged-in user
POST	/api/users/create	Yes	ADMIN	Admin creates user
PUT	/api/users/change-password/{id}	Yes	Any	Change password
GET	/api/users/export	Yes	ADMIN	Download users CSV

Query params for `/all`: `search`, `role`, `status`, `startDate`, `endDate`, `sortBy`, `sortDir`, `page` (default 1), `limit` (default 10).

Products (/api/products)

Method	Path	Auth	Role	Description
POST	/api/products/add	Yes	ADMIN, MANAGER	Create product (multipart)
PUT	/api/products/update	Yes	ADMIN, MANAGER	Update product (multipart)
GET	/api/products/all	Yes	Any	Paginated product list
GET	/api/products/{id}	Yes	Any	Get product by ID
DELETE	/api/products/delete/{id}	Yes	ADMIN, MANAGER	Soft-delete product
GET	/api/products/search	Yes	Any	Search by name/SKU
GET	/api/products/export	Yes	ADMIN, MANAGER	Download products CSV

Create/update use `multipart/form-data`. Fields: `name`, `price`, `stockQuantity`, `categoryId`, `reorderThreshold` (optional), `warehouseId` (optional), `description` (optional), `imageFile` (optional). Update also requires `productId`.

Query params for `/all`: `categoryId`, `warehouseId`, `minPrice`, `maxPrice`, `stockStatus` (`LOW`/`OUT`), `search`, `page` (default 1), `size` (default 10), `sortBy`, `sortDir`.

Categories (/api/categories)

Method	Path	Auth	Role	Description
POST	/api/categories/add	Yes	ADMIN, MANAGER	Create category
GET	/api/categories/all	Yes	Any	Paginated category list
GET	/api/categories/{id}	Yes	Any	Get category by ID
PUT	/api/categories/update/{id}	Yes	ADMIN, MANAGER	Update category
DELETE	/api/categories/delete/{id}	Yes	ADMIN, MANAGER	Delete category
PUT	/api/categories/toggle-status/{id}	Yes	ADMIN, MANAGER	Toggle ACTIVE/INACTIVE
GET	/api/categories/export	Yes	ADMIN	Download categories CSV

Suppliers (/api/suppliers)

Method	Path	Auth	Role	Description
POST	/api/suppliers/add	Yes	ADMIN, MANAGER	Create supplier
GET	/api/suppliers/all	Yes	Any	Paginated supplier list
GET	/api/suppliers/{id}	Yes	Any	Get supplier by ID
PUT	/api/suppliers/update/{id}	Yes	ADMIN, MANAGER	Update supplier
DELETE	/api/suppliers/delete/{id}	Yes	ADMIN, MANAGER	Delete supplier
PUT	/api/suppliers/toggle-status/{id}	Yes	ADMIN, MANAGER	Toggle ACTIVE/INACTIVE
GET	/api/suppliers/export	Yes	ADMIN	Download suppliers CSV

Warehouses (/api/warehouses)

Method	Path	Auth	Role	Description
GET	/api/warehouses/all	Yes	Any	Paginated warehouse list
GET	/api/warehouses/{id}	Yes	Any	Get warehouse by ID
POST	/api/warehouses/add	Yes	ADMIN, MANAGER	Create warehouse
PUT	/api/warehouses/update/{id}	Yes	ADMIN, MANAGER	Update warehouse
DELETE	/api/warehouses/delete/{id}	Yes	ADMIN, MANAGER	Delete warehouse
PUT	/api/warehouses/toggle-status/{id}	Yes	ADMIN, MANAGER	Toggle ACTIVE/INACTIVE
GET	/api/warehouses/export	Yes	ADMIN	Download warehouses CSV

Transactions (/api/transactions)

Method	Path	Auth	Role	Description
POST	/api/transactions/purchase	Yes	MANAGER, STAFF	Purchase (STAFF → proposal)
POST	/api/transactions/sell	Yes	MANAGER, STAFF	Sell (STAFF → proposal)
POST	/api/transactions/return	Yes	MANAGER	Return to supplier
GET	/api/transactions/all	Yes	Any	Paginated transaction list
GET	/api/transactions/{id}	Yes	Any	Get transaction by ID
GET	/api/transactions/by-month-year	Yes	Any	Transactions by month/year
PUT	/api/transactions/{id}	Yes	ADMIN, MANAGER	Update transaction status
GET	/api/transactions/export	Yes	ADMIN, MANAGER	Download transactions CSV

Purchase/sell body: `{"productId": 1, "supplierId": 2, "quantity": 10, "description": "..."}`

Status update body: a raw JSON string, e.g. `"COMPLETED"` (not an object).

Proposals (/api/proposals)

Method	Path	Auth	Role	Description
GET	/api/proposals/all	Yes	MANAGER, ADMIN	Paginated proposal list
GET	/api/proposals/{id}	Yes	MANAGER, ADMIN	Get proposal by ID
POST	/api/proposals/{id}/approve	Yes	MANAGER	Approve proposal
POST	/api/proposals/{id}/reject	Yes	MANAGER	Reject proposal
GET	/api/proposals/pending-count	Yes	MANAGER, ADMIN	Count of PENDING proposals
GET	/api/proposals/export	Yes	MANAGER	Download proposals CSV

Notifications (/api/notifications)

Method	Path	Auth	Role	Description
GET	/api/notifications	Yes	Any	Current user's notifications
GET	/api/notifications/unread-count	Yes	Any	Unread notification count
GET	/api/notifications/stream	Yes	Any	SSE event stream
PUT	/api/notifications/{id}/read	Yes	Any	Mark one as read
PUT	/api/notifications/read-all	Yes	Any	Mark all as read
DELETE	/api/notifications/all	Yes	Any	Delete all notifications
DELETE	/api/notifications/{id}	Yes	Any	Delete one notification

Audit Logs (/api/audit-logs)

Method	Path	Auth	Role	Description
GET	/api/audit-logs/all	Yes	ADMIN	Paginated audit log list
GET	/api/audit-logs/{id}	Yes	ADMIN	Get audit log by ID
GET	/api/audit-logs/export	Yes	ADMIN	Download audit logs CSV

Dashboard (/api/dashboard)

Method	Path	Auth	Role	Description
GET	/api/dashboard/stats	Yes	Any	Aggregated dashboard metrics
GET	/api/dashboard/export	Yes	ADMIN, MANAGER	Download dashboard stats CSV

Product–Supplier Links (/api/product-suppliers)

Method	Path	Auth	Role	Description
GET	/api/product-suppliers/{productId}	Yes	Any	Suppliers for a product
POST	/api/product-suppliers	Yes	ADMIN, MANAGER	Link supplier to product
PUT	/api/product-suppliers	Yes	ADMIN, MANAGER	Update link (cost/preferred)
DELETE	/api/product-suppliers/{pId}/{sId}	Yes	ADMIN, MANAGER	Remove link

Common Error Response Format

```
// 400 Bad Request (insufficient stock)
{ "status": 400, "message": "Insufficient stock. Available: 5",
  "timestamp": "2025-05-31T10:22:00" }

// 401 Unauthorized
{ "status": 401, "message": "JWT token is expired or invalid",
  "timestamp": "2025-05-31T10:22:00" }

// 403 Forbidden
{ "status": 403, "message": "Access Denied",
  "timestamp": "2025-05-31T10:22:00" }

// 404 Not Found
{ "status": 404, "message": "Product Not Found",
  "timestamp": "2025-05-31T10:22:00" }
```

Appendix C: Git Statistics

Commit graphs Since the complete Git commit graph is too extensive to display clearly within this document, we have exported it as a high-resolution PDF. To examine the full branching and merge history, please click here to view the detailed Commit Graph.

Contribution statistics per member

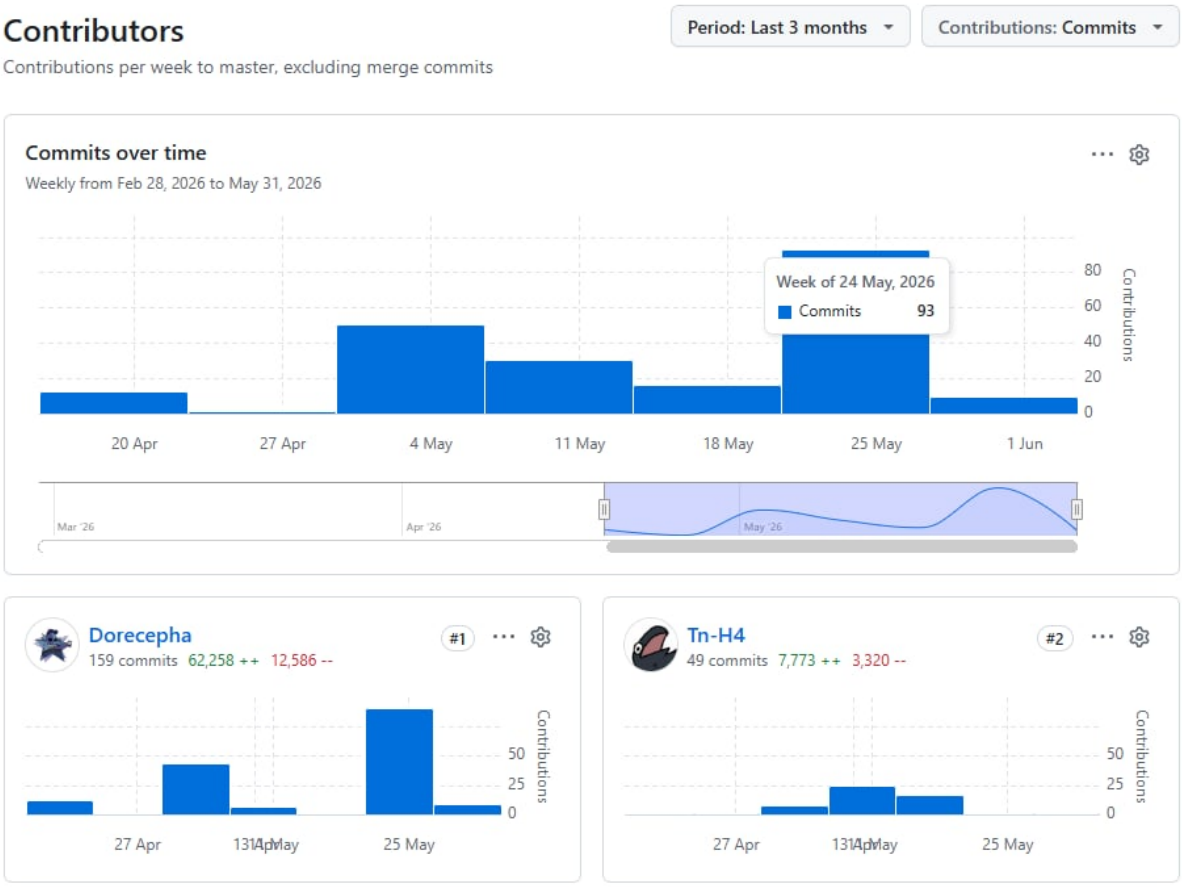


Figure 16: Contribution statistics

Appendix D: User Feedback

No formal external user testing was conducted for this academic project. The system was evaluated exclusively by the development team using the manual Black-Box test scenarios described in Section 6.1.